

# 樹莓派 GPIO 控制 — 使用 Python

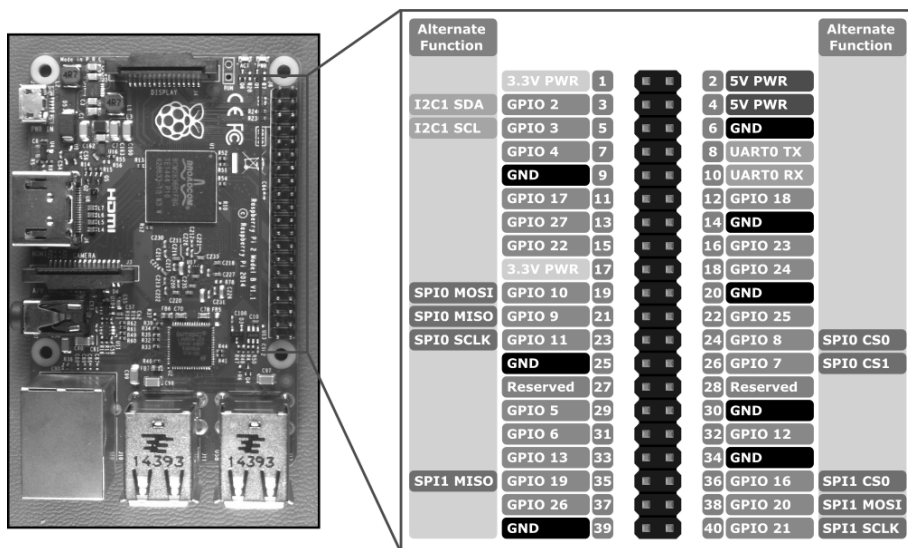
## 本章重點

- 9.1 Raspberry Pi 安裝 GPIO 模組
- 9.2 第一個 Raspberry Pi GPIO 的程式
- 9.3 Raspberry Pi GPIO 數位讀出 GPIO.BCM-LED
- 9.4 Raspberry Pi GPIO 數位讀出 GPIO.BOARD-LED
- 9.5 Raspberry Pi GPIO 的數位輸出——閃爍
- 9.6 Python GPIO 數位讀取——按鈕
- 9.7 Raspberry Pi GPIO 數位邊緣觸發
- 9.8 Raspberry PiGPIO 的 PWM 輸出
- 9.9 Raspberry Pi Analog 類比輸出
- 9.10 Raspberry Pi Analog 類比輸入
- 9.11 UART 序列埠資料傳遞—— 使用 UART 登入樹莓派
- 9.12 UART 序列埠資料傳遞——樹莓派和 PC
- 9.13 UART 序列埠資料傳遞——透過 Python
- 9.14 Raspberry Pi 的 GPIO SPI

## 9.1

## Raspberry Pi 安裝 GPIO 模組

樹莓派的 GPIO 最主要的目的，是在與周邊的電子零件進行資料的互動，例如控制 LED 的開關、讀取按鈕的是否被按下、何讀取感應器的數據等。



| Alternate Function |          |    |  |    | Alternate Function |
|--------------------|----------|----|--|----|--------------------|
|                    | 3.3V PWR | 1  |  | 2  | 5V PWR             |
| I2C1 SDA           | GPIO 2   | 3  |  | 4  | 5V PWR             |
| I2C1 SCL           | GPIO 3   | 5  |  | 6  | GND                |
|                    | GPIO 4   | 7  |  | 8  | UART0 TX           |
|                    | GND      | 9  |  | 10 | UART0 RX           |
|                    | GPIO 17  | 11 |  | 12 | GPIO 18            |
|                    | GPIO 27  | 13 |  | 14 | GND                |
|                    | GPIO 22  | 15 |  | 16 | GPIO 23            |
|                    | 3.3V PWR | 17 |  | 18 | GPIO 24            |
| SPI0 MOSI          | GPIO 10  | 19 |  | 20 | GND                |
| SPI0 MISO          | GPIO 9   | 21 |  | 22 | GPIO 25            |
| SPI0 SCLK          | GPIO 11  | 23 |  | 24 | GPIO 8             |
|                    | GND      | 25 |  | 26 | GPIO 7             |
|                    | Reserved | 27 |  | 28 | Reserved           |
|                    | GPIO 5   | 29 |  | 30 | GND                |
|                    | GPIO 6   | 31 |  | 32 | GPIO 12            |
|                    | GPIO 13  | 33 |  | 34 | GND                |
| SPI1 MISO          | GPIO 19  | 35 |  | 36 | GPIO 16            |
|                    | GPIO 26  | 37 |  | 38 | GPIO 20            |
|                    | GND      | 39 |  | 40 | GPIO 21            |
|                    |          |    |  |    | SPI0 CS0           |
|                    |          |    |  |    | SPI0 CS1           |
|                    |          |    |  |    | SPI1 CS0           |
|                    |          |    |  |    | SPI1 MOSI          |
|                    |          |    |  |    | SPI1 SCLK          |

圖 9-1 Raspberry Pi 2、3、4 的 GPIO 的接腳和用途



**注意** 這個表對樹莓派 GPIO 非常重要，請注意每一個接腳和編號。

因為 Raspberry Pi 實在太紅了，所以 Python 官方已經正式的把 Raspberry Pi 的 GPIO 功能正式加入 Python 中，所以本節的步驟不用特別執行。

而 Raspberry Pi 的 GPIO 函數因為持續更新，所以可不定時更新 GPIO 模組的版本。官方網址是 <https://pypi.python.org/pypi/RPi.GPIO>。

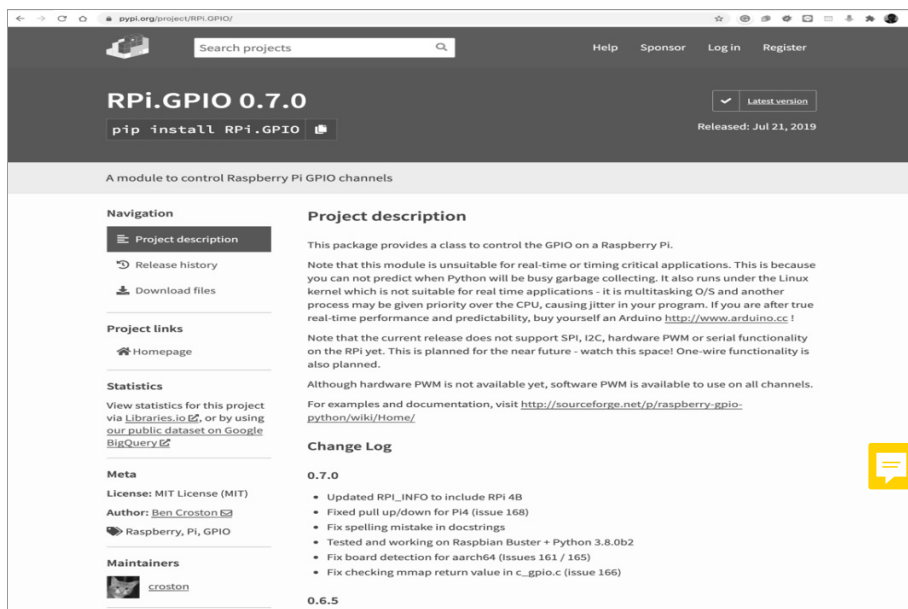


圖 9-2 Python RPi.GPIO 的下載網站

目前為止這個函式庫只有支援

- 數位輸入
- 數位資料輸出
- 使用軟體來模擬 PWM 的輸出

目前到 0.7.0 版本，還不能做到

- Serial 資料輸出和輸入
- I2C 資料輸出和輸入
- SPI 資料輸出和輸入
- PWM 硬體輸出入，但可以用軟體產生 PWM 軟體輸出。

此模組是開放原始程式碼 open source MIT 的專案，這些未完成的功能也許在不久的將來獲得改善，而本書會透過其他的函式庫來完成這些功能，請讀者放心。

因為 Raspberry Pi 的最新作業系統已經包含了此模組，若要取得最新的 RPi.GPIO，請透過以下的指令更新。

## 🌀 安裝 RPi.GPIO 流程

### STEP 1 更新 PIP。

透過以下指令更新 Python 的 PIP 軟體。

```
$ /usr/bin/python3 -m pip install --upgrade pip
```

```
pi@powenko:~ $ /usr/bin/python3 -m pip install --upgrade pip
Defaulting to user installation because normal site-packages is not writeable
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Collecting pip
  Downloading pip-20.3.3-py2.py3-none-any.whl (1.5 MB)
    |████████████████████| 1.5 MB 861 kB/s
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 20.2.4
    Uninstalling pip-20.2.4:
      Successfully uninstalled pip-20.2.4
^I$Successfully installed pip-20.3.3
```

圖 9-3 更新 PIP

### STEP 2 安裝 RPi.GPIO。

透過 PIP 工具將下載的 RPi.GPIO。

```
$ pip3 install RPi.GPIO
```

```
pi@powenko:~ $ pip3 install RPi.GPIO
Defaulting to user installation because normal site-packages is not writeable
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Requirement already satisfied: RPi.GPIO in /usr/lib/python3/dist-packages (0.7.0)
```

圖 9-4 安裝 RPi.GPIO

### STEP 3 確認版本。

透過 PIP3 的指令就確認 Python3 的 RPi.GPIO 版本。

```
$ pip3 list
```

```
pi@powenko:~ $ pip3 list
Package                               Version
-----
altgraph                             0.17
appdirs                              1.4.4
asn1crypto                           1.4.0
astroid                               2.4.2
asttokens                             2.0.4
automationhat                        0.2.2
backcall                             0.2.0
beautifulsoup4                       4.9.3
blinker                               1.4
blinky                                0.1.2
buttonshim                           0.0.2
Cap1xx                                0.1.3
certifi                               2020.12.5
```

圖 9-5 確認版本

往下找一下函數庫的名稱「RPi.GPIO」就能看到版本編號。

```
RPi.GPIO                             0.7.0
RTIMULib                             7.2.1
scrollphat                           0.0.7
scrollphathd                          1.2.1
SecretStorage                         2.3.1
semver                                2.0.1
Send2Trash                            1.5.0
sense-emu                             1.1
sense-hat                             2.2.0
setuptools                            40.8.0
simplegeneric                          0.8.1
simplejson                             3.16.0
six                                   1.12.0
skywriter                             0.0.7
sn3218                                1.2.7
soupsieve                             1.8
spidev                                3.4
ssh-import-id                         5.7
thonny                                3.2.6
tornado                               5.1.1
touchphat                             0.0.1
traitlets                             4.3.2
```

圖 9-6 RPi.GPIO 版本編號安裝 RPi.GPIO 模組

### 📺 教學影片

請見 9-1\_python-gpio-install.mp4 影片檔。

## 9.2

## 第一個 Raspberry Pi GPIO 的程式

## ● 介紹

使用 Python 來撰寫第一個 GPIO 的程式，為了使讀者知道原理，一樣分成二個方法來撰寫。

## ● 硬體準備

-  Raspberry Pi 板子。
-  1 個 LED 燈。
-  1 個 10K 的電阻（棕黑橘）。
-  麵包板。
-  電線數條。

## ● 硬體接線

將 Raspberry Pi 關機並拔除電源，依照下圖 9-7 的硬體接線圖，重新接一次硬體線路。確定 LED 燈短的接腳接到 Pin 6／Ground（就是 GND 接地），長的 LED 燈接到 Pin 7／GPIO4 後，下指令即可控制。

接著將 Raspberry Pi 的 GND 接地和 GPIO4（pin7）接上延長線到麵包板上，並接上 LED 燈和 1K 電阻。

| Raspberry Pi 接腳 pin | 元件接腳                                   |
|---------------------|----------------------------------------|
| Pin 7／GPIO4         | LED 燈的長腳                               |
| GND                 | LED 燈的短腳，先接 1K ohm 電阻，再接到 Raspberry Pi |

🔗 硬體接線圖：sample\ch09\blink.fzz

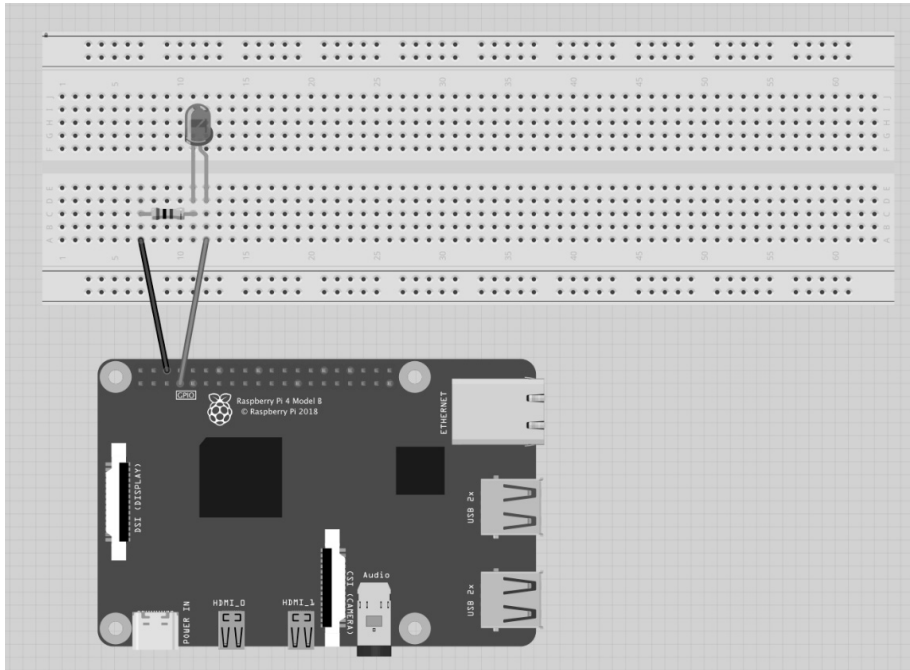


圖 9-7 實際硬體接線

❌ 直接在 Python 環境底下撰寫 **步驟**

❌ **步驟**

**STEP 1** 執行 Python3 的程式。

一樣透過文字模式，先執行 Python3 的程式。

```
$ python3
>>>
```

**STEP 2** 在 Python 中輸入 RPi.GPIO 模組。

輸入 RPi.GPIO 模組，代表可以控制 GPIO 周邊設備，並把以下內容輸入 Python 的環境。

```
>>> import RPi.GPIO as GPIO
>>> GPIO.setmode(GPIO.BCM)
>>> GPIO.setwarnings(False)
```

**STEP 3** 設定 Raspberry Pi 的 GPIO 接腳。

簡單的測試設定是否正確，設定接腳 GPIO4 是輸出還是輸入。

```
>>> GPIO.setup(4,GPIO.OUT)
```

**STEP 4** 控制 Raspberry Pi 接腳 GPIO4 的電位。

將接腳 4 的電位設定為高的話，只要下這個指令就可以。

```
>>> GPIO.output(4,1)
```

將接腳 4 的電位設定為低的話，則下這個指令。

```
>>> GPIO.output(4,0)
```

這個步驟 4 重複多少次都沒關係，每執行一次，就可以控制這個接腳的開和關的動作。

```
pi@powenko:~ $ python3
Python 3.7.3 (default, Dec 20 2019, 18:57:59)
[GCC 8.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import RPi.GPIO as GPIO
>>> GPIO.setmode(GPIO.BCM)
>>> GPIO.setwarnings(False)
>>> GPIO.setup(4,GPIO.OUT)
>>> GPIO.output(4,1)
>>> █
```

圖 9-8 整個動作流程



**注意** 請注意 Raspberry Pi 的 GPIO 輸出電壓是 3.3V，不是 5V。建議使用三用電表量一下，增加印象。

**STEP 5** 離開 Python。

透過以下的指令即可離開 Python 的環境。

```
>>> exit()
```

完整指令可以參考範例程式 `sample\ch09\01-gpio.py`

### 執行結果

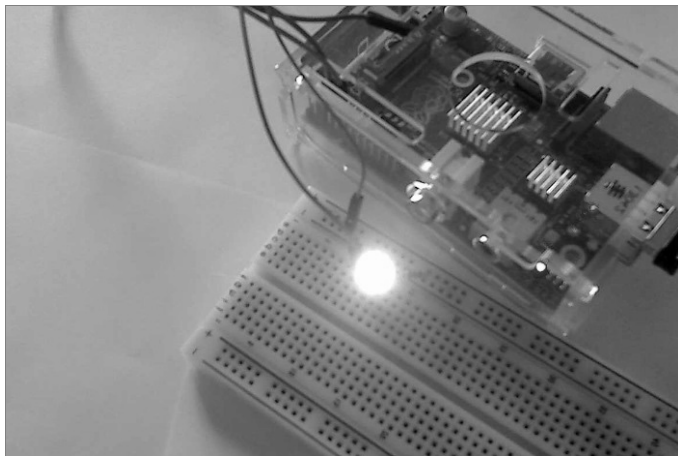


圖 9-9 實際執行結果

### 教學影片

請見 `9-2-python3-gpio.mp4` 影片檔（請注意程式中設定接腳的方法是 `GPIO.BCM`，所以這裡指定的 4 是 `GPIO4`，所以算起來是第七個接腳）。

## 9.3

### Raspberry Pi GPIO 數位讀出 GPIO.BCM-LED

### 步驟

**STEP 1** 打開文字編輯軟體，撰寫 Python 的程式。

透過文字編輯軟體 `nano`，來撰寫 Python 的程式。

```
$ nano 02-blink-GPIO4.py
```

**STEP 2** 撰寫程式。

這個程式主要的目的是要讓 GPIO4 同樣也是接腳 pin7，做開的動作 1 秒，再做關閉的動作 1 秒，一直重複。

 **範例程式：sample\ch09\02-blink-GPIO4.py**

```
1. #!/usr/bin/env python
2. # author: Powen Ko
3. import time, RPi.GPIO as GPIO
4. GPIO.setwarnings(False)           # 關閉 GPIO 警告訊息
5. GPIO.setmode(GPIO.BCM)           # 定義為 GPIO.BCM
6. GPIO.setup(4, GPIO.OUT)           # GPIO4 的接口為數位輸出
7.
8. while True:
9.     LEDon = GPIO.output(4, 0)     # GPIO4 暗
10.    time.sleep(1)
11.    LEDoff = GPIO.output(4, 1)     # GPIO4 亮
12.    time.sleep(1)
```

把程式複製和貼上，並透過「Ctrl + O」鍵儲存程式，最後透過「Ctrl + X」鍵離開程式。

 **程式解說**

- 第 1 行：這是執行環境的定義檔，定義 Python 程式放在哪個路徑，沒有寫也不會影響執行結果。
- 第 5 行：請注意，這裡是定義 GPIO.BCM。
- 第 8 行：迴圈開始。
- 第 9 行：關閉 GPIO4 上的電源。
- 第 10 行：休息一秒鐘。
- 第 11 行：打開 GPIO4 上的電源。



**注意** GPIO.setmode(GPIO.BCM) 意思是說後面的接腳數字 4，全部都是指 GPIO4 也就是接腳 pin7。



```
GNU nano 2.2.6 File: blink4.py

#!/usr/bin/env python
# author: Powen Ko
import time, RPi.GPIO as GPIO

GPIO.setmode(GPIO.BCM)
GPIO.setup(4, GPIO.OUT)

while True:
    LEDon = GPIO.output(4, 0)
    time.sleep(1)
    LEDoff = GPIO.output(4, 1)
    time.sleep(1)
```

圖 9-10 整個動作流程

## ◎ 執行結果

同上一章節。

### — 教學影片 —

請見 9-3\_demo-python-blink-GPIO4.mp4 影片檔(該影片介紹如何透過 Python 控制 GPIO、Raspberry Pi 接腳 GPIO4 的電源輸出)。

## 9.4

## Raspberry Pi GPIO 數位讀出 GPIO.BOARD-LED

GPIO.setmode 有兩種用法：

1. GPIO.setmode ( GPIO.BCM )
2. GPIO.setmode ( GPIO.BOARD )

它們的差異在於：

- **GPIO.BCM**：程式中 GPIO 模組的數字，定義方法會是以 1 為 GPIO1、2 為 GPIO2、3 為 GPIO3 等。
- **GPIO.BOARD**：程式中 GPIO 接腳號碼，定義方法會是以 1 為接腳 pin 1、2 為接腳 pin 2、3 為接腳 pin 3 等。

定義的方法不一樣，所控制的輸出也會不一樣。

如果用 `GPIO.setmode(GPIO.BOARD)` 撰寫，在硬體接線如上一個章節完全一樣，不用調整，完整的程式如下。

### 範例程式：sample\ch09\03-blink-Pin7.py

```

1. #!/usr/bin/env python
2. # author: Powen Ko
3. import time, RPi.GPIO as GPIO
4. GPIO.setwarnings(False)
5. GPIO.setmode(GPIO.BOARD)          # 設定為 BOARD
6. GPIO.setup(7, GPIO.OUT)           # Pin7 為數位輸出
7.
8. while True:
9.     LEDon = GPIO.output(7, 0)    # Pin 4 暗
10.    time.sleep(1)
11.    LEDoff = GPIO.output(7, 1)    # Pin 4 亮
12.    time.sleep(1)

```

### 教學影片

請見 9-4 *python-blink-Pin7.mp4* 影片檔（該影片介紹如何透過 Python 控制 GPIORaspberry Pi 接腳 Pin7 的電源輸出，並且透過開、關的動作，製造 LED 燈閃爍的效果）。

## 9.5

# Raspberry Pi GPIO 的數位輸出——閃爍

### ◎ 介紹

透過程式做修改，讓 LED 閃爍四次之後，就會離開程式。

### ◎ 步驟

透過 nano 文字編輯器輸入以下的程式，並存成 blink.py。此處筆者是使用 nano 文字編輯器。

```
$nano 04-blink4Times.py
```

並把下面的程式複製上去。

### ◎ 範例程式：sample\ch09\04-blink4Times.py

```
1. import RPi.GPIO as GPIO
2. import time
3. GPIO.setwarnings(False)
4. GPIO.setmode(GPIO.BCM)
5. GPIO.setup(4,GPIO.OUT)
6. count=0
7. while (count<4):           # 迴圈判斷
8.     GPIO.output(4, 1)      # GPIO4 亮
9.     time.sleep(1)          # 休息一秒
10.    GPIO.output(4, 0)      # GPIO4 暗
11.    time.sleep(1)          # 休息一秒
12.    count=count+1          # 計數器變數做累加的動作
13.
14. print("Good bye powenko.com")
```

把程式複製和貼上，並透過「Ctrl + O」鍵儲存程式，最後透過「Ctrl + X」鍵離開程式。

### ◎ 程式解說

- 第 1 行：這是執行環境的定義檔，定義 Python 程式放在哪個路徑。
- 第 5 行：請注意，這裡是定義 GPIO.BCM。

- 第 7 行：迴圈開始，由 0 到 3。
- 第 9 行：休息一秒鐘。
- 第 12 行：迴圈結尾。
- 第 14 行：顯示文字，就像是 `print` 的功用，把 "" 之中的文字顯示出來。

## 執行結果

如何單獨執行 Python 程式：

可以透過以下指令執行 Python，這裡使用的 Python 程式檔案名稱是 `blink.py`，如果讀者要儲存成不同的檔案名稱，直接修改檔名即可。

```
$ sudo chmod +x blink.py  
$sudo ./blink.py
```



## 教學影片

請見 9-5 *blink4Times.mp4* 影片檔（該影片介紹如何透過 Python 控制檔案的存取並改變 GPIO 接腳的電源輸出，讓燈光閃爍 4 次後結束程式）。

## 9.6

## Python GPIO 數位讀取——按鈕

### 介紹

一般來說，在撰寫控制周邊時，除了數位資料輸出之外，還有一個很重要的功能——讀取數位資料的輸入。這個功能可以用在很多地方，例如：實際的硬體按鈕是否被按下、感應器現在的執行情況等的應用。

底下將帶領讀者做一個硬體實驗，增加一個按鈕，並透過程式來判斷使用者是否按下，且當使用者按下按鈕時，程式便會得到使用者按下按鈕的動作，並且將 LED 燈亮起來。

## 硬體準備

-  Raspberry Pi 板子。
-  1 個 LED 燈。
-  2 個 10K 的電阻（棕黑橘）。
-  1 個 4 個接腳的按鈕。
-  麵包板。
-  電線數條。

## 硬體接線

請先將 Raspberry Pi 關機並拔除電源，依照下圖的硬體接線圖，重新接一次硬體線路。LED 燈短的接腳接到 Ground，長的 LED 燈接腳則接到 GPIO4（即 Pin 7），Raspberry Pi 的 GND 接地和 GPIO4/ Pin 7 接上延長線到麵包板，並接上 LED 燈和 1K 電阻，到目前為止都跟前面範例的硬體接線相同。

接著將 4 個接腳的按鈕接到麵包板，底下的 2 個接腳分別接上電阻和電源，按鈕上頭的接腳，接上 Raspberry Pi 的 GPIO17 / Pin 11 的接腳。

| Raspberry Pi 接腳 pin | 元件接腳                                   |
|---------------------|----------------------------------------|
| Pin 7/GPIO4         | LED 燈的長腳                               |
| GND                 | LED 燈的短腳，先接 1K ohm 電阻，再接到 Raspberry Pi |
| Pin 11/GPIO17       | 按鈕上頭的接腳                                |
| GND                 | 按鈕底下的 1 個接腳接上電阻                        |
| Pin 2/5V            | 按鈕底下的另一個接腳接上電源                         |

### 硬體接線圖：sample\ch09\05-input.fzz

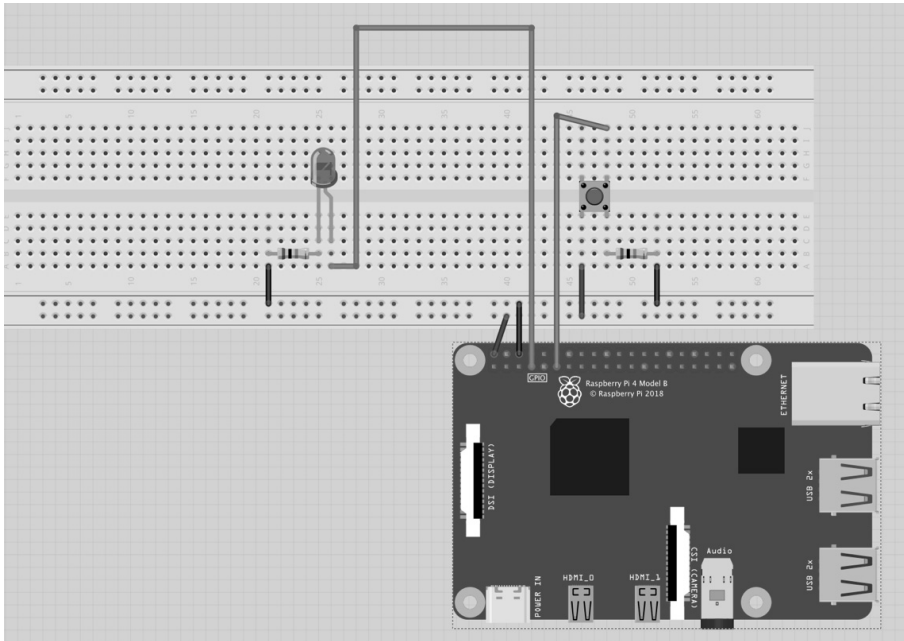


圖 9-11 實際硬體接線

透過文字編輯器輸入以下的程式，並存成 `input.py`，此處筆者是使用 `nano` 文字編輯器。

```
$nano 05-input.py
```

並把下面的程式複製上去。

### 範例程式：sample\ch09\05-input.py

```
1. #!/usr/bin/python
2. import RPi.GPIO as GPIO
3. import time
4. GPIO.setwarnings(False)
5. GPIO.setmode(GPIO.BCM)
6. GPIO.setup(4,GPIO.OUT)
7. GPIO.setup(17,GPIO.IN)
8. while True:
9.     value01=GPIO.input(17)      # 讀取電壓回傳 1 或 0
10.    GPIO.output(4,value01)      # 輸出
11.    time.sleep(0.1)
```

把程式複製和貼上，並透過「Ctrl + O」鍵儲存程式，最後透過「Ctrl + X」鍵離開程式。

### 🔗 程式解說

- 第 1 行：註解的寫法，前面加個# 就可以成為註解。
- 第 4 行：關閉 GPIO 的警告訊息。
- 第 5 行：請注意，這裡是定義 GPIO.BCM。
- 第 6 行：設定 GPIO4 / Pin 7 的接腳，負責資料輸出 GPIO.OUT。
- 第 7 行：設定 GPIO17 / Pin 11 的接腳，負責資料輸入 GPIO.IN。
- 第 8 行：迴圈開始。
- 第 9 行：讀取 GPIO17 / Pin 11 的資料輸入。
- 第 10 行：設定 GPIO4 / Pin 7 的資料輸出。
- 第 11 行：休息 0.1 秒鐘。

### 🔗 執行結果

透過以下的指令即可執行 Python 程式。

```
$python305-input.py
```

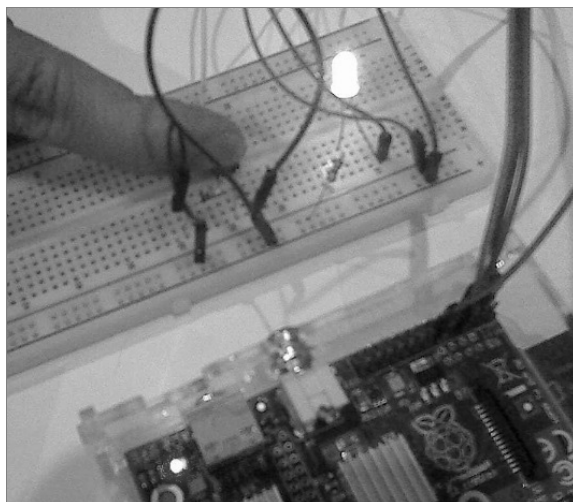


圖 9-12 執行結果

第 9 行 `GPIO.input(17)` 的電壓，讀到的是高過 3.3V 的話就會回傳 1，不然就會回傳 0，所以按下按鈕時，LED 燈就會亮。

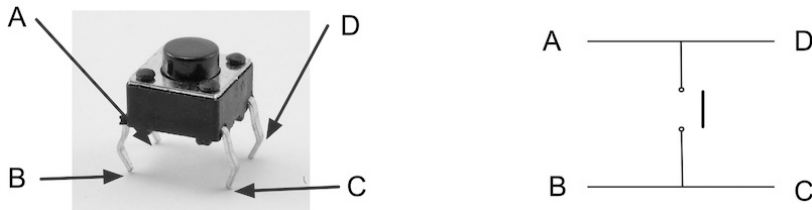


圖 9-13 4 個接腳的按鈕內的硬體線路

而 4 個接腳的按鈕：

- A 接腳會連到樹莓派的 GPIO4 / Pin 7 的接腳。
- D 接腳會連到樹莓派的 GND / Pin 6 的接腳。
- C 接腳會連到樹莓派的 5V / Pin 2 的接腳。

而按鈕平常沒有壓下時：

- 因為 A 和 D 相接，所以平時沒按下時，GPIO4 / Pin 7 的接腳會讀到 GND 的值，所以為 0。

而按鈕壓下時：

- 因為 A、C 和 D 相接，所以按下時，GPIO4 / Pin 7 的接腳會讀到 5V 的值，所以為 1。

#### 教學影片

請見 8-3-4 `python-input.mp4` 影片檔（該影片介紹如何透過 Python 控制按鍵的動作，去調整 LED 的開關）。

## 9.7

# Raspberry Pi GPIO 數位邊緣觸發

### ◎ 介紹

數位輸入的動作，除了按下和放開之外，還有一組按下和放開那一剎那的動作。

在寫程式之前，先介紹數位輸入的幾個函數：

- `wait_for_edge`（接腳號碼）
- `event_detected`（接腳號碼）
- `add_event_detect`（接腳號碼，動作）
- `add_event_callback`（接腳號碼，呼叫的函數）

這是用來偵測輸入的變化，`edge` 動作有三種：

- `GPIO.RISING`：低到高。
- `GPIO.FALLING`：高到低。
- `GPIO.BOTH`：低到高和高到低這兩者的動作。

### ◎ `GPIO.wait_for_edge(channel, edge)`

等到事件發生時，才繼續往下一行程式執行。

- `Channel`：接腳。
- `edge`：偵測三種輸入的變化 `GPIO.RISING`（低到高），`GPIO.FALLING`（高到低），`GPIO.BOTH`（低到高和高到低動作）。

### ◎ 使用範例

`GPIO4 / Pin 7` 的接腳，當按鍵壓下去時，才能夠繼續執行。

```
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)
GPIO.setup(12, GPIO.OUT)
GPIO.wait_for_edge(4, GPIO.RISING)
```

### 🔗 GPIO.add\_event\_detect(channel)

偵測三種輸入變化的接腳。

- channel：接腳。

### 🔗 回傳值=GPIO.event\_detected(edge)

詢問是否發生了三種輸入變化的函數。

- edge：偵測三種輸入的變化 GPIO.RISING（低到高），GPIO.FALLING（高到低），GPIO.BOTH（低到高和高到低動作）。
- 回傳值：True 和 False，判斷是否發生。

### 🔗 使用範例

GPIO4 / Pin 7 的接腳，當按鍵放開時，會列印出 Button pressed 字串，可以透過 add\_event\_detect，設定 GPIO.RISING。

```
1. #!/usr/bin/python
2. #Author: Powen Ko
3. import RPi.GPIO as GPIO
4.
5. GPIO.setmode(GPIO.BCM)
6. GPIO.setup(4,GPIO.IN)
7.
8. GPIO.add_event_detect(4, GPIO.RISING)
9. while True:
10.     if GPIO.event_detected(4):
11.         print('Button pressed')
```

### 🔗 GPIO.add\_event\_callback(edge, DefName)

用來定義當特定的觸發事件發生時，要呼叫哪一個函數。

- edge：偵測三種輸入的變化 GPIO.RISING（低到高），GPIO.FALLING（高到低），GPIO.BOTH（低到高和高到低動作）。
- DefName：呼叫的函數。

## 🔗 使用範例

當按鍵壓下去時，做出反應，GPIO17 / Pin 11 的接腳，當按鍵壓下去時，會列印出「This is a edge event callback function!」字串。

```
1. #!/usr/bin/python
2. import RPi.GPIO as GPIO
3.
4. GPIO.setmode(GPIO.BCM)
5. GPIO.setup(17,GPIO.IN)
6. def my_callback(channel):
7.     print('This is a edge event callback function!')
```



這範例特別的是透過 `GPIO.add_event_callback`，當事件發生時，會呼叫 `y_callback()` 函數。

在這裡要分享，如果是 C 語言的函數，會寫成如下

```
void my_callback(int channel){
    print('This is a edge event callback function!');
}
```



而 Python 函數的寫法則如下所示，透過相同的空白，來帶表示同一個 `{...}` 的範圍。

```
my_callback(channel):
    print('This is a edge event callback function!')
```



## 🔗 硬體準備

-  Raspberry Pi 板子。
-  2 個 10K 的電阻（棕黑橘）。
-  2 個 4 個接腳的按鈕。
-  麵包板。
-  電線數條。

## 硬體接線

請先將 Raspberry Pi 關機並拔除電源，依照圖 9-14 的硬體接線圖，重新接一次硬體線路。

| Raspberry Pi 接腳 pin | 元件接腳                                        |
|---------------------|---------------------------------------------|
| Pin 7 / GPIO4       | 第一個按鍵的 pin 4                                |
| Pin 11/ GPIO17      | 第二個按鍵的 pin 4                                |
| Pin 6/ GND          | 第一個按鍵的 pin 3，先接 10K ohm 電阻，再接到 Raspberry Pi |
| Pin 6/ GND          | 第二個按鍵的 pin 3，先接 10K ohm 電阻，再接到 Raspberry Pi |
| Pin 2/ 5V           | 第一個按鍵的 pin 1                                |
| Pin 2/ 5V           | 第二個按鍵的 pin 1                                |

## 硬體接線圖：sample\ch09\06-edge.fzz

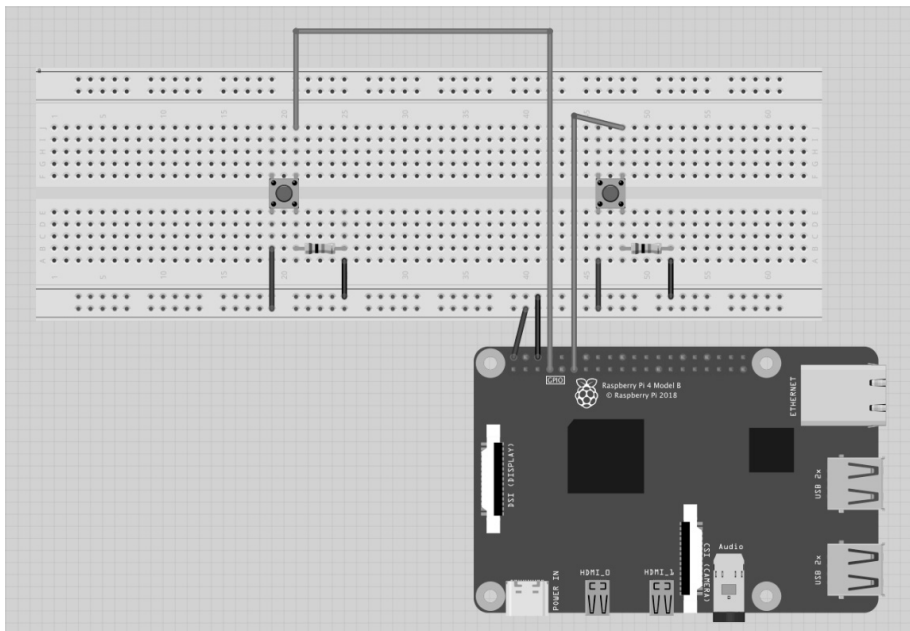


圖 9-14 實際硬體接線

透過文字編輯器輸入以下的程式，並存成 `edge.py`，此處筆者是使用 `nano` 文字編輯器。

```
$nano edge.py
```

並把程式複製上去。

### 範例程式：sample\ch09\06-edge.py

```
1. #!/usr/bin/python
2. #Author: Powen Ko
3. import RPi.GPIO as GPIO
4. import time
5. GPIO.setwarnings(False)
6. GPIO.setmode(GPIO.BCM)
7. GPIO.setup(4,GPIO.IN)
8. GPIO.setup(17,GPIO.IN)
9. def my_callback(channel):
10.     print('This is a edge event callback function!')
11.
12. GPIO.add_event_detect(17, GPIO.FALLING)
13. GPIO.add_event_callback(17,my_callback)
14.
15. GPIO.add_event_detect(4, GPIO.RISING)
16. while True:
17.     if GPIO.event_detected(4):
18.         print('Button pressed')
19.
20.
```

把程式複製和貼上，並透過「Ctrl + O」鍵儲存程式，最後透過「Ctrl + X」鍵離開程式。

### 程式解說

- 第 1 行：這是執行環境的定義檔，定義 Python 程式放在哪個路徑。
- 第 2 行：註解的寫法，前面加個# 就可以成為註解。
- 第 6 行：請注意，這裡是定義 `GPIO.BCM`。
- 第 7 行：設定 GPIO 4 / Pin 7 的接腳，負責資料輸出 `GPIO.IN`。
- 第 8 行：設定 GPIO 17 / Pin 11 的接腳，負責資料輸入 `GPIO.IN`。

- 第 12-13 行：設定當 GPIO17 的接腳按鍵放開時，就會產生 FALLING 電位下降的訊號，透過 callback 函數就會呼叫 my\_callback 函數。
- 第 15-18 行：設定當 GPIO4 的接腳按鍵壓下時，就會顯示 Button pressed 的字串。

## 🎯 執行結果

透過以下的指令即可執行 Python 程式。

```
$ python3 06-edge.py
```



```
pi@raspberrypi ~ $ python3 06-edge.py
This is a edge event callback function!
This is a edge event callback function!
Button pressed
Button pressed
Button pressed
Button pressed
Button pressed
This is a edge event callback function!
Button pressed
This is a edge event callback function!
Button pressed
This is a edge event callback function!
Button pressed
```

圖 9-15 執行結果

## 📺 教學影片

請見 9-6\_python-gpio-edge.mp4 影片檔（該影片將介紹如何透過 Python 取得 GPIO 接腳的「按下去」、「放開」的那一剎那）。

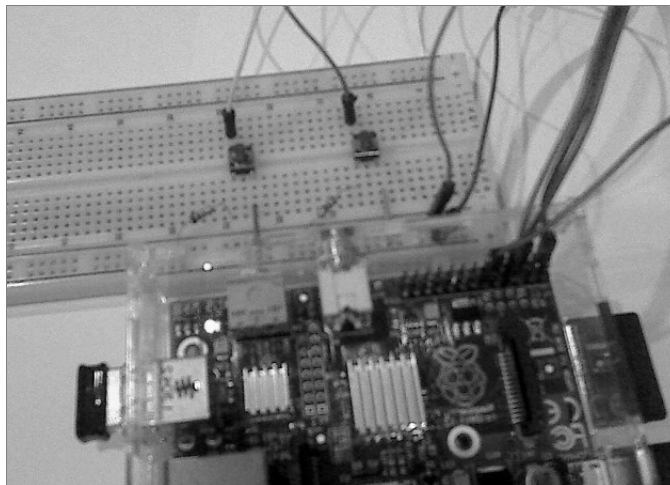


圖 9-16 實際硬體

## 9.8

### Raspberry Pi GPIO 的 PWM 輸出

#### 介紹

脈衝寬度調製（Pulse Width Modulation，PWM）是利用微處理器的數位輸出，來模擬電路進行控制的一種非常有效的技術，被廣泛應用在測量、通信、步進馬達、功率控制變換等。

PWM 透過對一系列脈衝的寬度進行調配，來有效地獲得所需要的波形和電壓。它是一種類比時間高低控制方式，根據相應的變化，調製電晶體柵極或基極的偏置，來實現開關穩壓電源輸出電晶體或電晶體導通時間的改變。這種方式能使電源的輸出，在工作條件變化時產生不同的電壓，這是利用微處理器的數位輸出，來控制類比電路的一種非常有效的功能。

什麼是 PWM 訊號？它是將訊號編碼於脈波寬度上的一種技術，此技術以數位方式來模擬類比訊號，廣泛應用在資料傳輸上。因數位訊號只存在 High、LOW 電位的變化，相較於類比訊號，比較不會受到雜訊干擾。PWM 訊號中，脈波寬度在整個週期所占的比例稱為工作週期（duty cycle），是指位於邏輯高準位（logic high level）的波型在整個週期中所占的比例。

PWM 的一個優點是從處理器到被控系統信號都是數位形式的，無需進行轉換。讓信號保持為數位形式可使資料穩定，而這也是將 PWM 用於通信的主要原因。從模擬信號轉向 PWM 可以極大地延長通信距離，在接收端，透過適當的 RC 或 LC 網路可以濾除調製高頻方波，並將信號還原為模擬形式。

PWM 控制技術一直是變頻技術的核心技術之一。1964 年 A.Schonung 和 H.Stemmler 首先提出，將這項通訊技術應用到交流傳動中。從最初採用模擬電路完成三角調製波和參考正弦波比較，如下圖所示，把類比的電壓轉換為數位 PWM 的電壓型態。

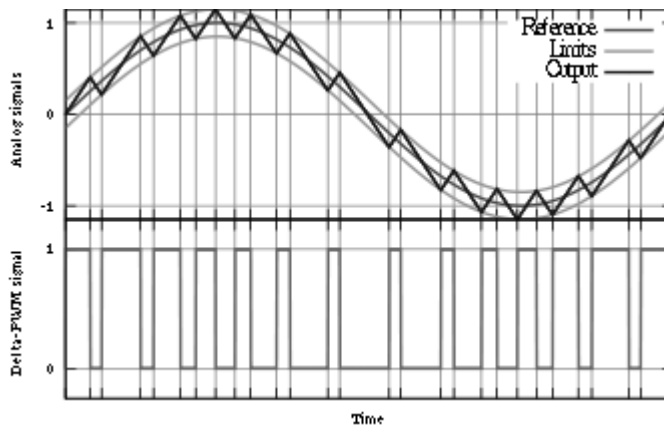


圖 9-17 類比轉成 PWM

嚴格來說，硬體上並沒有提供一個叫 PWM 的設計，Raspberry Pi 上的 PWM 功能是透過軟體模擬出來的，可以借用 GPIO.PWM 的函數來達到這樣的功能。

## 函數介紹

### GPIO.PWM(channel, frequency)

該函數實際執行的原理是透過自己定的頻率。

- channel：接腳號碼。
- frequency：頻率單位是 Hz。

### 使用範例

設定接腳 4 的頻率為 100 Hz。

```
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)
GPIO.PWM(4, 100)
```

### GPIO.start(dc)

於程式啟動 PWM 時使用。

- dc：duty cycle 範圍在 0.0 到 100.0 之間，意思是高電壓的時間，在這個頻率中占多少。

### 使用範例

設定接腳 4 的頻率為 100 Hz，並且設定一半是高電壓，一半是低電壓。

```
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)
GPIO.PWM(4, 100)
GPIO.start(50)
```

### GPIO.changeFrequency(frequency)

程式中即時改變 frequency 頻率單位是 Hz，意思是一秒內變換幾次。

- frequency：頻率單位是 Hz。

### 使用範例

設定接腳 4 的頻率為 50 Hz，由 100% 都是高電壓，變換到頻率為 20 Hz。

```
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)
GPIO.PWM(4, 50)
GPIO.start(100)
GPIO.start(50)
```

## GPIO.changeDutyCycle(dc)

- dc：duty cycle 範圍在 0.0 到 100.0 之間，意思是高電壓的時間，在這個頻率中占多少。

## 使用範例

設定接腳 4 的頻率為 100 Hz，由 100% 都是高電壓，變換到一半是高電壓，一半是低電壓。

```
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)
GPIO.setup(12, GPIO.OUT)
GPIO.PWM(4, 100)
GPIO.start(100)
GPIO.ChangeDutyCycle(50)
```

如果設定 PWM 是 500Hz，下圖 9-18 的粗黑直線每個間隔就是 2 Milliseconds，即 1/500Hz 秒，也就是 PWM 的極限。

透過 GPIO.start(dc) 函數，指定 dc 參數數值在 0 到 100 之間，所以 GPIO.start(100) 100% 都是高電壓，造成 3.3V 的效果，若 GPIO.start(100/2)，就會造成 1.65V 的效果。

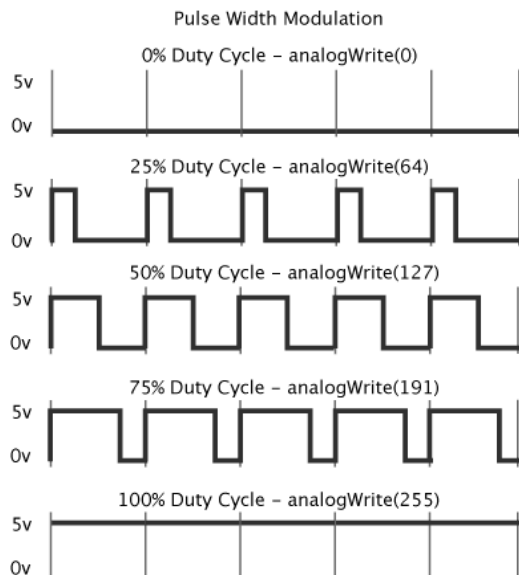


圖 9-18 類比轉成 PWM

如果有機會看視波器上輸出的樣子，如圖 9-19 所示。那是高電壓 3.3V 和 0V 之間快速轉換，透過時間差比例，才模擬出類比輸出這樣的效果。

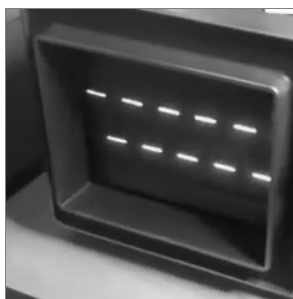


圖 9-19 PWM 的範例，在視波器上執行的情況

本章節會透過 Python GPIO 來寫一個 PWM 的函數，以下範例就直接透過 LED 燈來顯示送出去的情況。

### ● 硬體準備

-  Raspberry Pi 板子。
-  2 個 10K 的電阻（棕黑橘）。
-  2 個 LED 燈。
-  麵包板。
-  電線數條。

### ● 硬體接線

先將 Raspberry Pi 關機並拔除電源，依照圖 9-20 的硬體接線圖，重新接一次硬體線路。

| Raspberry Pi 接腳 pin | 元件接腳                                        |
|---------------------|---------------------------------------------|
| Pin 7 / GPIO4       | 第一個 LED 燈的長腳                                |
| Pin 11/ GPIO17      | 第二個 LED 燈的長腳                                |
| Pin 6/ GND          | 第一個 LED 燈的短腳，先接 10K ohm 電阻，再接到 Raspberry Pi |
| Pin 6/ GND          | 第二個 LED 燈的短腳，先接 10K ohm 電阻，再接到 Raspberry Pi |

硬體接線圖：sample\ch09\07-pwm.fzz

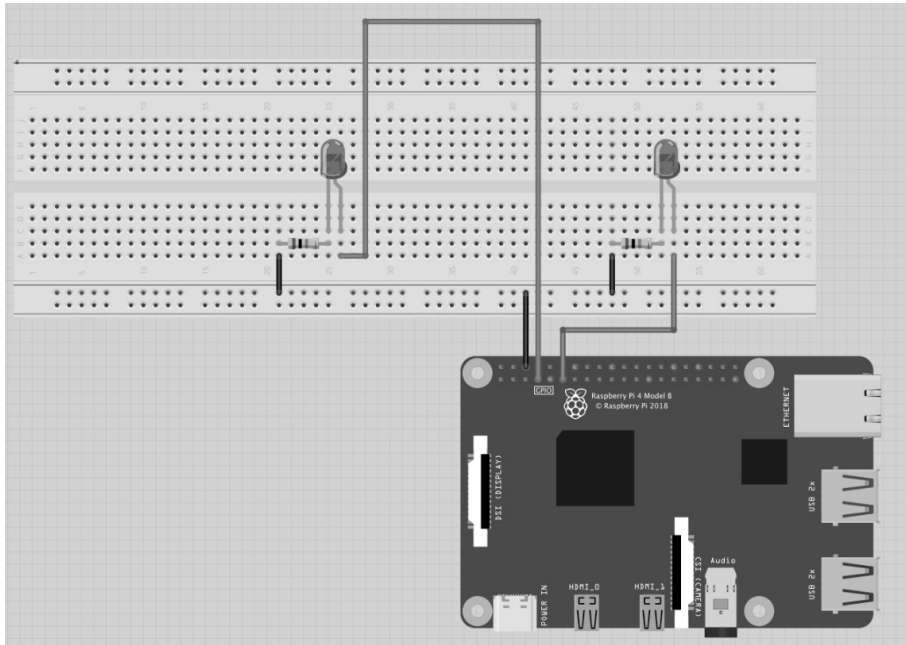


圖 9-20 實際硬體接線

透過文字編輯器輸入以下的程式，並儲存為 `pwm.py`，此處筆者是使用 `nano` 文字編輯器。

```
$nano pwm.py
```

並把程式複製上去。

範例程式：sample\ch09\07-pwm.py

```
1. #!/usr/bin/env python
2. # author: Powen Ko
3. import time, RPi.GPIO as GPIO
4. GPIO.setwarnings(False)
5. GPIO.setmode(GPIO.BCM)
6. GPIO.setup(4, GPIO.OUT)
7. GPIO.setup(17, GPIO.OUT)
8.
9. p1 = GPIO.PWM(4,1000) # GPIO4 頻率=1000Hz
10. p2 = GPIO.PWM(17,10) # GPIO17 頻率=10Hz
11. p1.start(0)           # 0% 的高電壓
```

```
12. p2.start(50)           # 50% 的高電壓
13.
14. while True:
15.     for dc in range (5,101,5):
16.         p1.changeDutyCycle(dc)
17.         p2.changeFrequency(dc)
18.         time.sleep(0.2);
19.
```

把程式複製和貼上，並透過「Ctrl + O」鍵儲存程式，最後透過「Ctrl + X」鍵離開程式。

### 🔗 程式解說

- 第 1 行：這是執行環境的定義檔，定義 Python 程式放在哪個路徑。
- 第 2 行：註解的寫法，前面加個# 就可以成為註解。
- 第 5 行：請注意，這裡是定義 GPIO.BCM。
- 第 6 行：設定 GPIO 4 / Pin 7 的接腳，負責資料輸出 GPIO.OUT。
- 第 7 行：設定 GPIO 17 / Pin 11 的接腳，負責資料輸出 GPIO.OUT。
- 第 16 行：調整 GPIO4 的 DutyCycle 開關的比例。
- 第 17 行：調整 GPIO4 的 Frequency 頻率的時間。

### 🔗 執行結果

透過以下的指令即可執行 Python 程式，會發現如下的情況。

- GPIO 4 / Pin 7 在高速的頻率下，調整 duty cycle，會造成 LED 燈的明亮差異。
- GPIO 17 / Pin 11 在改變 PWM 的頻率下，會造成 LED 閃爍的快慢。

```
$python307-pwm.py
```

```
pi@raspberrypi ~ $ python3 07-pwm.py
pwm.py:6: RuntimeWarning: This channel is already in use, continuing anyway.
  Use GPIO.setwarnings(False) to disable warnings.
  GPIO.setup(4, GPIO.OUT)
pwm.py:7: RuntimeWarning: This channel is already in use, continuing anyway.
  Use GPIO.setwarnings(False) to disable warnings.
  GPIO.setup(17, GPIO.OUT)
```

圖 9-21 執行結果

### 教學影片

請見 9-7 *\_python-gpio-pwm.mp4* 影片檔（該影片介紹如何透過 Python 改變 PWM 的頻率和 duty cycle 的比例，改變 PWM 的頻率會造成 LED 閃爍的快慢，而在高速的頻率下，調整 duty cycle 會影響 LED 燈的明亮差異。教學影片的效果絕對比圖片好，強烈建議讀者觀看）。

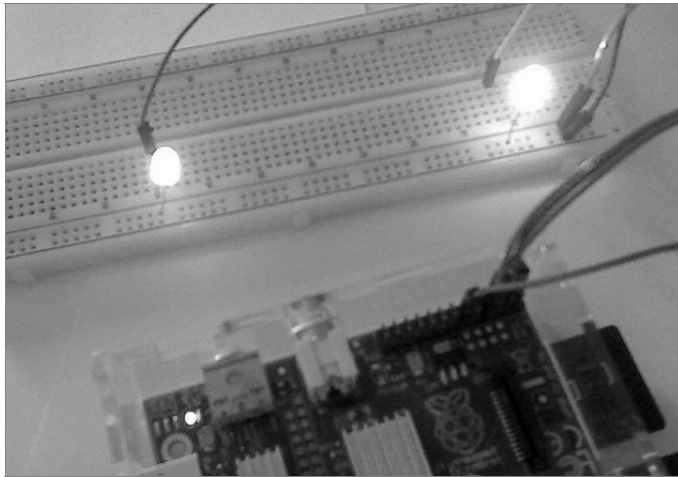


圖 9-22 實際硬體

## 9.9

### Raspberry Pi Analog 類比輸出

嚴格來說，Raspberry Pi 無法透過它現在的硬體 GPIO 做出輸出類比 Analog 電壓，但是在實際開發上有這樣需求的感應器實在太多，在此，將說明如何透過特別的技巧來達到此功能。

## 介紹

事實上 Raspberry Pi 無法直接輸出真正的類比電壓，像是 1.2V 或 0.8V 等這樣特定的電壓，那本節的類比輸出函數是如何做到？

### 方法一：

透過特別的 DAC (Digital to Analog Converter) IC 來做到這樣的效果。例如常見的 DAC IC 有：

- DAC0800LCN
- DAC080LCN
- DAC0808LCN
- TLC7524CD

### 方法二：

本節是透過 PWM 的方式在接腳上輸出一個高、一個低的效果，因為切換的速度極快，才會產生電壓差。

關於 PWM，在本書前面的 PWM 章節已介紹相關的知識，可以用這個方法模擬出類比電壓，AnalogWrite 較多被應用在 LED 亮度控制、電機轉速控制等方面。

可以透過高頻率的 PWM，並且讓高低電壓，對應的占空比（一定的時間中，低電壓的百分比）為 0 至 100%，所以輸出的電壓會在 0 到 3.3V 之間，所以是以  $3.3V \times \text{value} / 100$  這樣的公式，來達到電壓的輸出。

前面有簡單提過 PWM，這裡就不多說了，本節會透過改變類比電壓，來調整 LED 燈的明亮。

## 硬體準備

-  Raspberry Pi 板子。
-  1 個 10K 的電阻（棕黑橘）。
-  1 個 LED 燈。

- 麵包線
- 電線數條

-  麵包板。
-  電線數條。

### ◎ 硬體接線

先將 Raspberry Pi 關機並拔除電源，依照圖 9-23 的硬體接線圖，重新接一次硬體線路。

| Raspberry Pi 接腳 pin | 元件接腳                                        |
|---------------------|---------------------------------------------|
| Pin 7 / GPIO4       | 第一個 LED 燈的長腳                                |
| Pin 6/ GND          | 第一個 LED 燈的短腳，先接 10K ohm 電阻，再接到 Raspberry Pi |

### ◎ 硬體接線圖：sample\ch09\08-analog\_outA.fzz

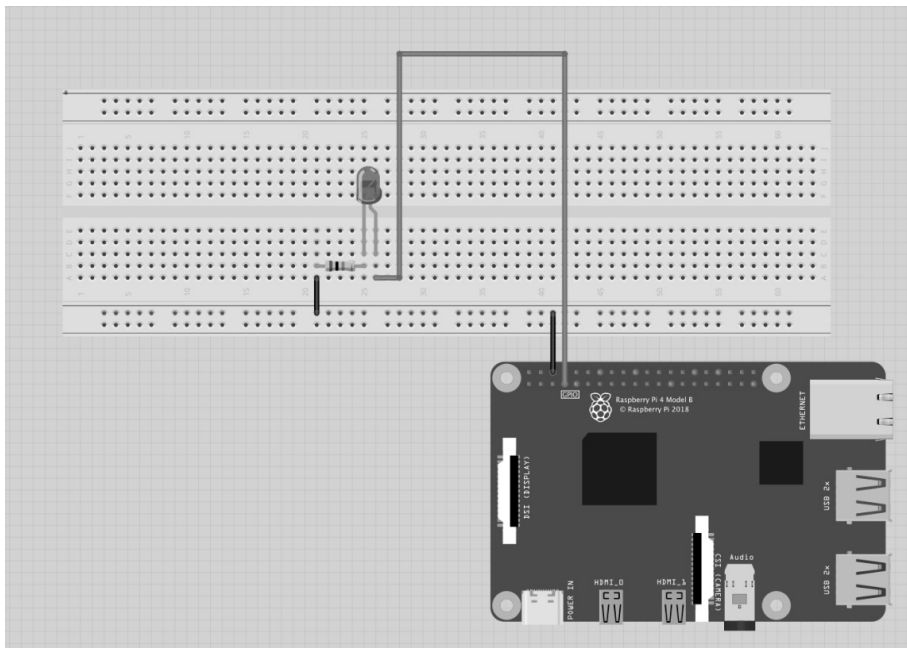


圖 9-23 實際硬體接線

但是，如果時間和設備允許，建議透過以下的硬體線路，把 PWM 的類比資料，透過電容的充電放電的功能，讓原本方方正正的類比訊號，轉換成有弧度類比訊號，這樣的硬體設計才會比較符合類比輸出。

所需硬體材料有：

- 1 個 3.9K 的電阻（橘白紅）。
- 1 個 0.1uF 電容。

● 硬體接線圖：sample\ch09\09-analog\_outB.fzz

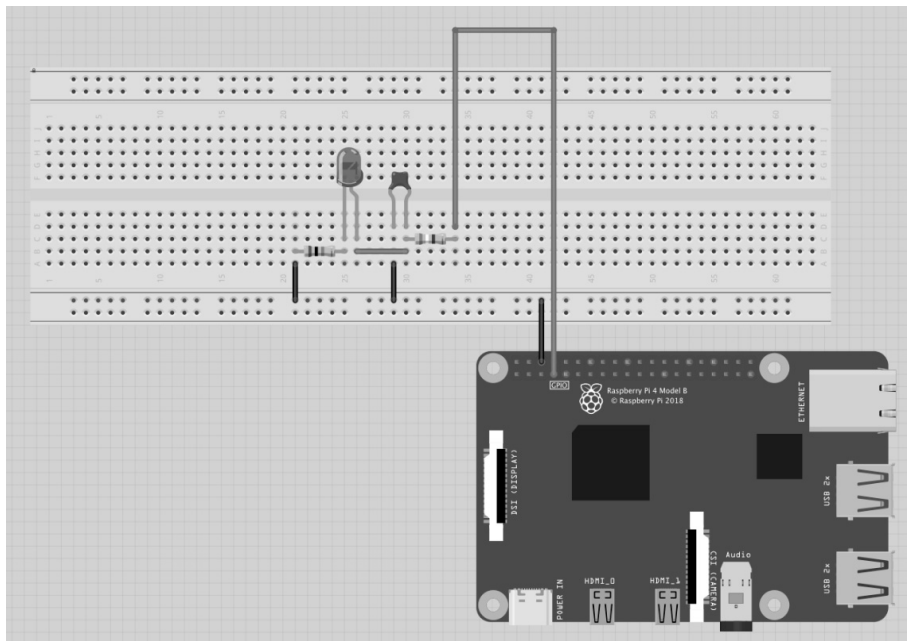


圖 9-24 加上 DAC 的實際硬體接線

如果設備許可，可以在視波器上面看到輸出的波形，會比較符合實際電壓，而不是高低電壓的變化，如下圖所示。

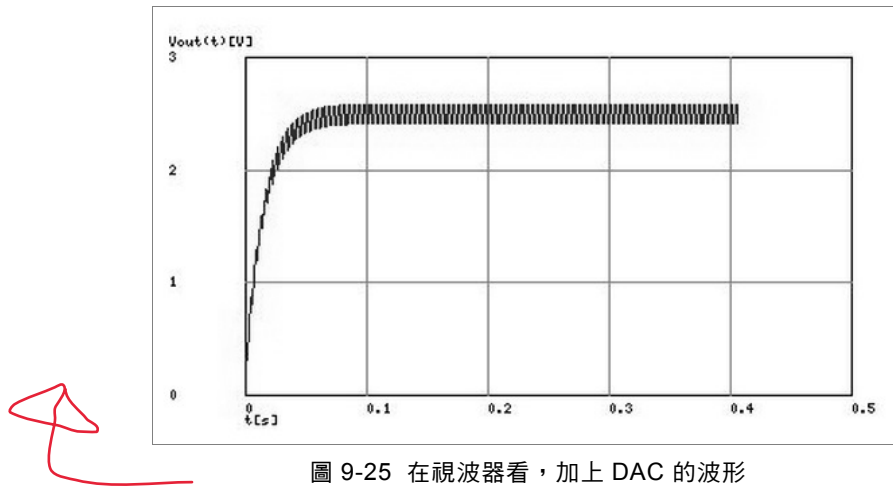


圖 9-25 在視波器看，加上 DAC 的波形

透過文字編輯器輸入以下的程式，並存成 `edge.py`，此處筆者是使用 `nano` 文字編輯器。

```
$nano 08-analog_output.py
```

並把程式複製上去。

#### 範例程式：sample\ch09\08-analog\_output.py

```
1. #!/usr/bin/env python
2. # author: Powen Ko
3. import time, RPi.GPIO as GPIO
4. GPIO.setwarnings(False)
5. GPIO.setmode(GPIO.BCM)
6. GPIO.setup(4, GPIO.OUT)
7.
8. p1 = GPIO.PWM(4,1000) # GPIO4 頻率=1000Hz
9. p1.start(0)           # 0% 的高電壓
10.
11. while True:
12.     for dc in range(10,101,5):
13.         p1.changeDutyCycle(dc) # 調整高電壓的比率
14.         time.sleep(1); # 暫停一秒
15.
16.
```

把程式複製和貼上，並透過「Ctrl + O」鍵儲存程式，最後透過「Ctrl + X」鍵離開程式。

### 程式解說

- 第 1 行：這是執行環境的定義檔，定義 Python 程式放在哪個路徑。
- 第 2 行：註解的寫法，前面加個# 就可以成為註解。
- 第 5 行：請注意，這裡是定義 GPIO.BCM。
- 第 6 行：設定 GPIO 4 / Pin 7 的接腳，負責資料輸出 GPIO.OUT。
- 第 13 行：改變週期的時間。

### 執行結果

透過以下的指令即可執行 Python 程式。

```
$ python3 08-analog_output.py
```

```
pi@raspberrypi ~ $ python3 08-analog_output.py
analog_output.py:6: RuntimeWarning: This channel is already in use, continuing anyway. Use GPIO.setwarnings(False) to disable warnings.
  GPIO.setup(4, GPIO.OUT)
```

圖 9-26 執行結果

### 教學影片

請見 *9-8\_python-gpio-analog.mp4* 影片檔（該影片介紹如何透過 Python 指定 GPIO 接腳的輸出電壓）。

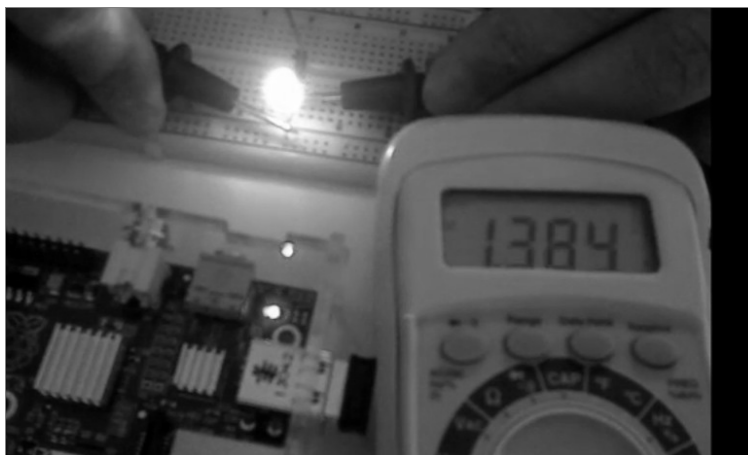


圖 9-27 實際硬體量測結果

## 9.10

## Raspberry Pi Analog 類比輸入

## ◎ 介紹

Raspberry Pi 無法直接輸入真正所謂的類比電壓，如 1.2V 或 0.8V 等，那有什麼方法可做到這樣的功能？因為有些感應器，是透過電壓改變回傳感應的情況。

這需要透過其他方法來解決這個問題，其中可以透過 ADC（Analog to digital converters）IC 來讀取類比資料，並透過 IC 轉換成 8bit 的數位資料。常見的 ADC IC 有：

- ADC0804：有八個接腳的數位輸出（本節會介紹）。
- MCP3008：只有四個接腳的數位輸出（本書 SPI 章節）。

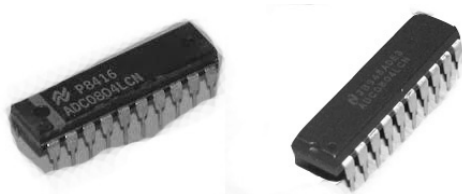


圖 9-28 ADC IC 的外觀



**提醒** 如果手上的 ADC IC 是 MCP3008，可以參考本書 SPI 章節，裡面有詳細的介紹。

但有趣的是，大多的 ADC IC 都是透過「SPI」的方法與 IC 溝通。

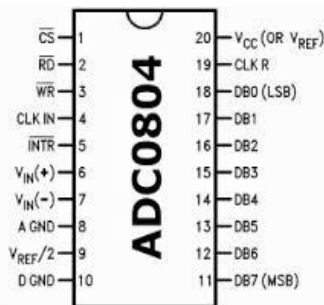


圖 9-29 接腳功能圖

## 接腳方法

| Pin 號碼 | 功能                                      | 名稱               |
|--------|-----------------------------------------|------------------|
| 1      | 啟動 ADC；低電位時才會工作。                        | CS (Chip Select) |
| 2      | 輸入端子；從高到低的脈衝後，會帶來讀取 8 個接腳轉換後的資料。        | RD (Read) 讀取     |
| 3      | 輸入接腳：當低到高電位時，就會開始進行資料轉換。                | Write            |
| 4      | 輸入時脈。                                   | CLK IN(Clock IN) |
| 5      | 輸出接腳；當這接腳輸出為低電壓時，就代表轉換完成。               | Interrupt        |
| 6      | 類比的非反相（non-inverting）輸入，也就是要量測的類比電壓的接腳。 | Vin(+)           |
| 7      | 類比的反向輸入，通常都直接接到 GND 接地上。                | Vin(-)           |
| 8      | 接地。                                     | Analog Ground    |
| 9      | 輸入接腳：設置的參考類比電壓的輸入。                      | Vref/2           |
| 10     | 接地。                                     | Digital Ground   |
| 11     | 8 bit 數位輸出接腳。                           | D7               |
| 12     |                                         | D6               |
| 13     |                                         | D5               |
| 14     |                                         | D4               |
| 15     |                                         | D3               |
| 16     |                                         | D2               |
| 17     |                                         | D1               |
| 18     |                                         | D0               |
| 19     | 提供內部計時器使用。                              | Clock R          |
| 20     | 5V 電壓。                                  | Vcc              |

實驗一

光是 ADC 0804 接線就已經很複雜了，所以筆者把這次的動作分成兩階段，先來測試 ADC 是否接的正確。首先，做一個實驗，讀取可變電阻的資料，並透過 LED 燈顯示出來，這樣就是一個標準的讀取電阻值的硬體。

● 硬體準備

-  1 個 10K 的電阻（棕黑橘）。
-  8 個 LED 燈。
-  1 個 IC 是 ADC0804。
-  1 個 150 pf 的電容。
-  1 個可變電阻。
-  麵包板。
-  電線數條。
- 

● 硬體接線

這個實驗不用 Raspberry Pi 的硬體和軟體，是單純的硬體測試，請在沒有接上電源和電池的情況下，依照圖 9-30 的硬體接線圖，接過一次硬體線路。理論上 LED 燈要接電阻以防止燒壞。如果時間允許建議這麼做，這裡因為線路已經很完善，也可以省略，建議買好一點的 LED 燈，避免燒壞 LED 燈。

| ADC0804 接腳 pin | 元件接腳                                             |
|----------------|--------------------------------------------------|
| Pin 1          | 電池 GND                                           |
| Pin 2          | 電池 GND                                           |
| Pin 3          | Pin 5                                            |
| Pin 4          | 接到電容                                             |
| Pin 5          | Pin 3                                            |
| Pin 6          | 接到可變電阻                                           |
| Pin 7          | 電池 GND                                           |
| Pin 8          | 電池 GND                                           |
| Pin 9          | 不用接線                                             |
| Pin 10         | 電池 GND                                           |
| Pin 11         | 第 1 個 LED 燈的長腳，然後 LED 燈的短腳，先接 10K ohm 電阻，再接到 GND |

| ADC0804 接腳 pin | 元件接腳                                             |
|----------------|--------------------------------------------------|
| Pin 12         | 第 2 個 LED 燈的長腳，然後 LED 燈的短腳，先接 10K ohm 電阻，再接到 GND |
| Pin 13         | 第 3 個 LED 燈的長腳，然後 LED 燈的短腳，先接 10K ohm 電阻，再接到 GND |
| Pin 14         | 第 4 個 LED 燈的長腳，然後 LED 燈的短腳，先接 10K ohm 電阻，再接到 GND |
| Pin 15         | 第 5 個 LED 燈的長腳，然後 LED 燈的短腳，先接 10K ohm 電阻，再接到 GND |
| Pin 16         | 第 6 個 LED 燈的長腳，然後 LED 燈的短腳，先接 10K ohm 電阻，再接到 GND |
| Pin 17         | 第 7 個 LED 燈的長腳，然後 LED 燈的短腳，先接 10K ohm 電阻，再接到 GND |
| Pin 18         | 第 8 個 LED 燈的長腳，然後 LED 燈的短腳，先接 10K ohm 電阻，再接到 GND |
| Pin 19         | 先接 10K ohm 電阻，再接到電容                              |
| Pin 20         | 3.3V 電池的正極                                       |

◎ 硬體接線圖：sample\ch09\10-analog\_inputA.fzz

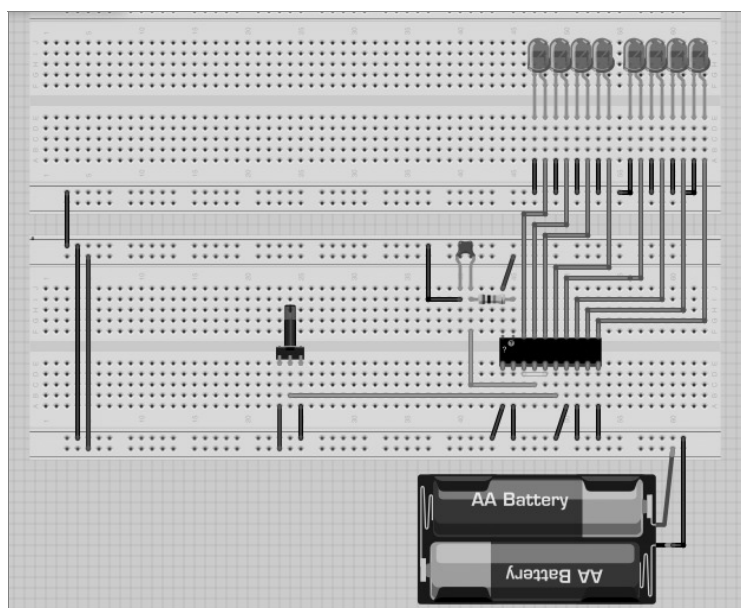


圖 9-30 實際硬體接線

## 執行結果

此實驗不用 Raspberry Pi，請先確定硬體線路接線正確後，再接上 5V 或 3.3V 的電池（2 個 AA 的電池）。實驗的目的是透過調整可變電阻，就可以看到 8 個 LED 燈顯示 2 進位的電阻資料。如果手邊沒有電池，也可以用 Raspberry Pi 的 GPIO 上的 Pin1 電壓為 3.3 V 和 Pin6 GND 來代替電池。

### 教學影片

由於硬體線路較多，如果操作不順利，可以和 [9-9\\_adc0804-demo.mp4](#) 影片檔互相比對，看是哪個接腳沒有接好。該影片介紹如何透過 ADC0804 的 IC 讀取可變電阻，並把資料顯示在 8 個 LED 上。

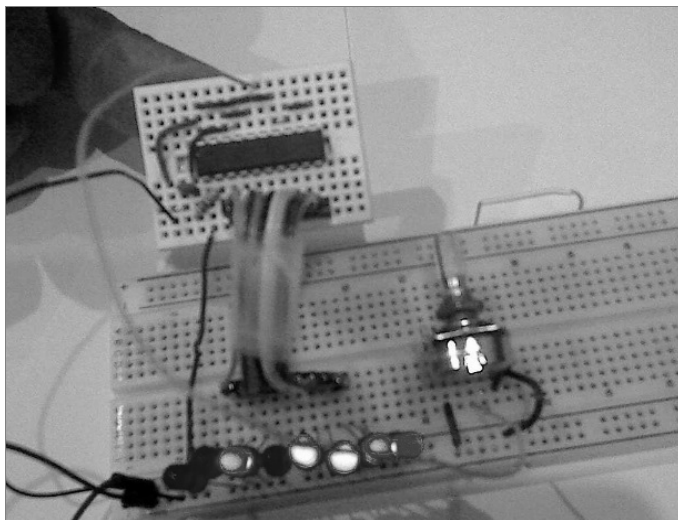


圖 9-31 實際硬體量測結果

## 實驗二

如果第一個實驗順利，接下來繼續做第二個實驗，把這 8 條所謂的數位資料接到 Raspberry Pi 的板子上，再透過程式來讀取資料，並把它顯示在電腦螢幕上。

### 硬體準備

跟上一個實驗一樣的硬體，請把電池移除，這裡使用 Raspberry Pi 來供電。

-  Raspberry Pi 板子。
-  電線數條。

### 硬體接線

請先把上一個 ADC0804 實驗接好，關機後拔除電源，並把上一個實驗的電池拔掉，依照圖 9-32 的硬體接線圖接上 Raspberry Pi 的硬體線路。唯一不同的是把 LED 燈上面的 8 條接線，接到 Raspberry Pi 的 GPIO 上，和電源做些調整就好。

| Raspberry Pi 接腳 pin | ADC0804 元件 |
|---------------------|------------|
| Pin 1 / 電源 3.3V     | Pin 20     |
| Pin 6 / GND         | 電源 GND     |
| Pin 7 / GPIO4       | Pin 18     |
| Pin 11 / GPIO17     | Pin 17     |
| Pin 12 / GPIO18     | Pin 16     |
| Pin 13 / GPIO21     | Pin 15     |
| Pin 15 / GPIO22     | Pin 14     |
| Pin 16 / GPIO23     | Pin 13     |
| Pin 18 / GPIO24     | Pin 12     |
| Pin 22 / GPIO25     | Pin 11     |

🔗 範例程式：sample\ch09\11-analog\_inputB.fzz

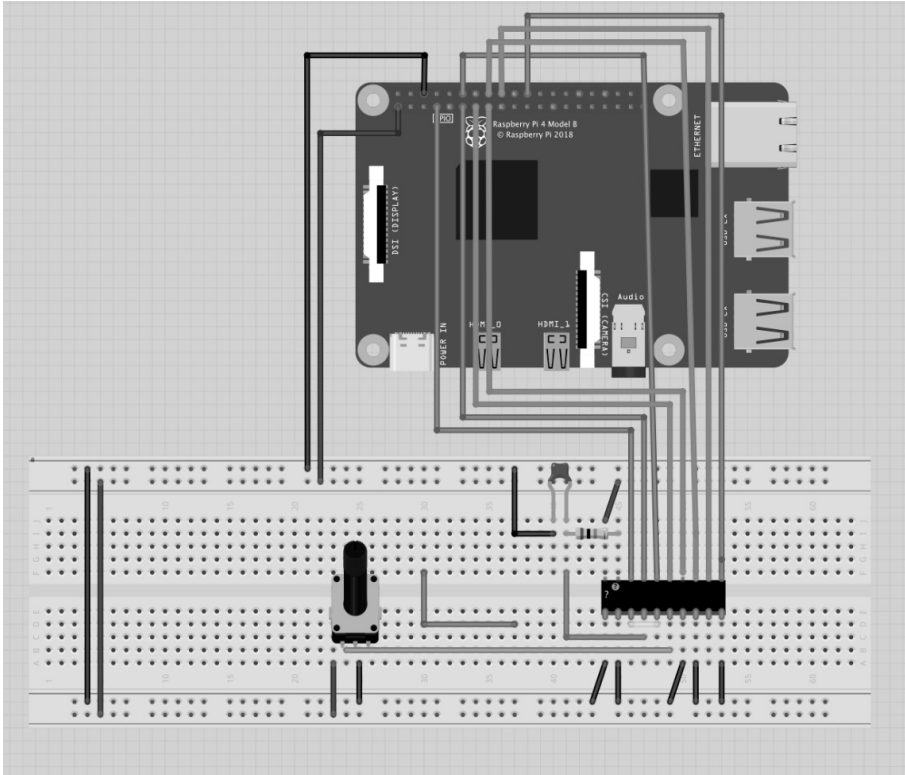


圖 9-32 實際硬體接線

透過文字編輯器輸入以下的程式，並存成 11-analog\_input.py，此處筆者是使用 nano 文字編輯器。

```
$nano 11-analog_input.py
```

並把程式複製上去。

🔗 範例程式：sample\ch09\11-analog\_input.py

```
1. #!/usr/bin/env python
2. # author: Powen Ko
3. import time, RPi.GPIO as GPIO
4. GPIO.setwarnings(False)
5. GPIO.setmode(GPIO.BOARD)      # 注意這裡用 BOARD
6. GPIO.setup(7, GPIO.IN)
7. GPIO.setup(11, GPIO.IN)
```

```
8. GPIO.setup(12, GPIO.IN)
9. GPIO.setup(13, GPIO.IN)
10. GPIO.setup(15, GPIO.IN)
11. GPIO.setup(16, GPIO.IN)
12. GPIO.setup(18, GPIO.IN)
13. GPIO.setup(22, GPIO.IN)
14. while True:
15.     a0 = GPIO.input(7)
16.     a1 = GPIO.input(11)
17.     a2 = GPIO.input(12)
18.     a3 = GPIO.input(13)
19.     a4 = GPIO.input(15)
20.     a5 = GPIO.input(16)
21.     a6 = GPIO.input(18)
22.     a7 = GPIO.input(22)
23.     total=a0+(a1*2)+(a2*4)+(a3*8)+(a4*16)+(a5*32)+(a6*64)+(a7*128)
24.     print(a7,a6,a5,a4,a3,a2,a1,a0,"[",total,"]")
25.     time.sleep(0.1)
```

把程式複製和貼上，並透過「Ctrl + O」鍵儲存程式，最後透過「Ctrl + X」鍵離開程式。

### 程式解說

- 第 1 行：這是執行環境的定義檔，定義 Python 程式放在哪個路徑。
- 第 2 行：註解的寫法，前面加個# 就可以成為註解。
- 第 5 行：請注意，這裡是定義 GPIO.BOARD，所以是用實際的接腳號碼來做設定。
- 第 6 行：設定 GPIO 4 / Pin 7 的接腳，負責資料輸入 GPIO.IN。
- 第 14 行：無盡迴圈。
- 第 15 行：讀取 GPIO 4 / Pin 7 的接腳，並存放在變數 a0。
- 第 23 行：因為每一個接腳的資料只有 1 和 0，所以換算 2 進位到 10 進位，方便閱讀。
- 第 24 行：顯示每個接腳的資料數入，和全部的 10 進位資料。

## 執行結果

透過以下的指令，就可以執行 Python 程式。要離開的時候按下「Ctrl + Z」鍵即可。

```
$ python3 11-analog_input.py
```

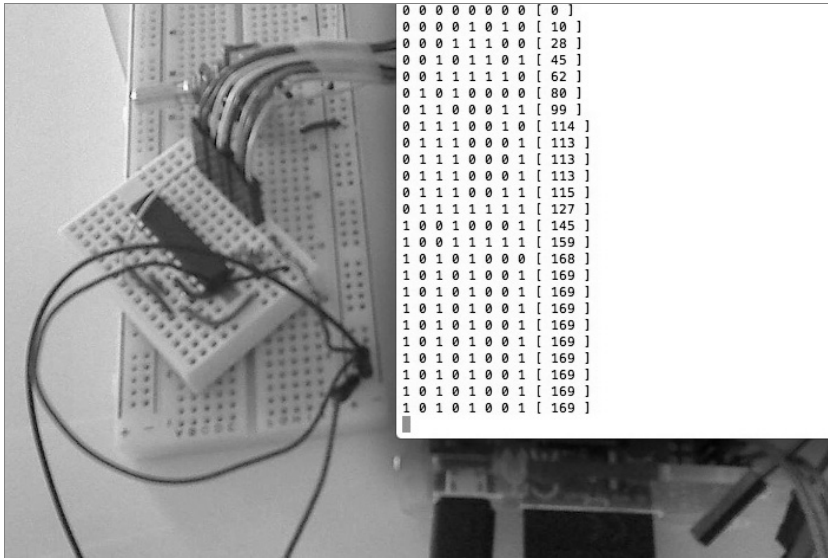


圖 9-33 執行結果

## 教學影片

請見 9-10\_python-gpio-analog-input.mp4 影片檔，該影片介紹如何透過 Python 讀取可變電阻的值。

### 補充資料

別擔心因為一下子占用了 8 個 GPIO 的接腳後，就沒有其他的接腳可以使用，萬一接腳還是不夠的話，有二個方法：

1. 透過 74HC595 的 IC 來達到。只需 Raspberry Pi 的 3 個 GPIO 的接腳。
2. 使用 Arduino 這類的板子。透過 Arduino 讀取類比資料輸入，而樹莓派和 Arduino 透過「Raspberry Pi 的 GPIO UART 序列埠資料傳遞」的技巧溝通。

## 9.11

### UART 序列埠資料傳遞—— 使用 UART 登入樹莓派

本節會介紹一種常見的資料傳輸技術，可以讓資料在不同的機器之間傳遞，這樣的技術普遍使用在電子業界，例如 Raspberry Pi 運用這樣的技術和 Arduino、PC 等其他硬體做溝通。

#### ◎ UART 是什麼？

UART 是通用異步收發傳輸器（Universal Asynchronous Receiver/Transmitter，通常稱作 UART），是一種異步收發傳輸器，為電腦硬體的一部分，將資料由串行通信與並行通信間作傳輸轉換。UART 通常用在與其他通訊介面（如 RS-232, COM port, TTL Serial）的連結上。

#### ◎ USB 轉 TTL 轉換器

筆者比較建議直接透過二個 USB 轉 TTL 的轉接頭，就能達到目的。因為 Linux 驅動程式的考量，建議使用 PL2303 這個 IC。

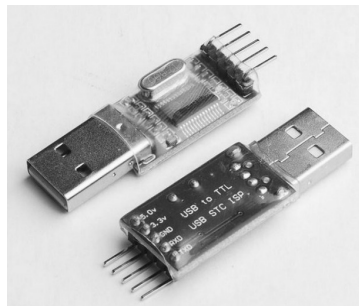


圖 9-34 USB 轉 com 的轉接頭

樹莓派雖然 GPIO 有一組 Pin8 和 Pin10 RX/TX 的 UART，但是太多軟體、系統版本問題，需要修改很多軟體檔案設定，所以除非真的要節省成本，可以找最新的設定方法來處理。筆者會建議直接添購 USB 轉 TTL 的硬體，接在樹莓派的 USB 上直接使用。

## ◎ 介紹

為了證明 Raspberry Pi 可以做 UART 的資料傳遞，需要借用個人電腦的序列埠，跟著整個實驗步驟到最後，就會看到如何做到個人電腦與 Raspberry Pi 的資料傳遞。

這裡 Raspberry Pi 和電腦連接：

- PC 使用 USB 轉 TTLSerial 硬體。
- RPI 使用 GPIO Pin 6,8,10 接到 TLLSerial 硬體。

## ◎ 硬體準備，PC 使用 RS243 轉 TTL

-  Raspberry Pi 板子。
-  1 個 COM 序列埠 RS232 轉 TTL 的轉換器。
-  麵包板。
-  接線。

## ◎ 硬體接線

在 PC 和樹莓派的 USB 接口，各接一個 USB 轉 TTL 並透過杜邦線，將兩者接在一起。

| Raspberry Pi 端 | P C 端 USB 轉 TTL Serial |
|----------------|------------------------|
| Pin10/RX       | USB 轉 TTL 板子的 TX       |
| Pin8/TX        | USB 轉 TTL 板子的 RX       |
| Pin6/GND 接地    | USB 轉 TTL 的 GND 接地     |

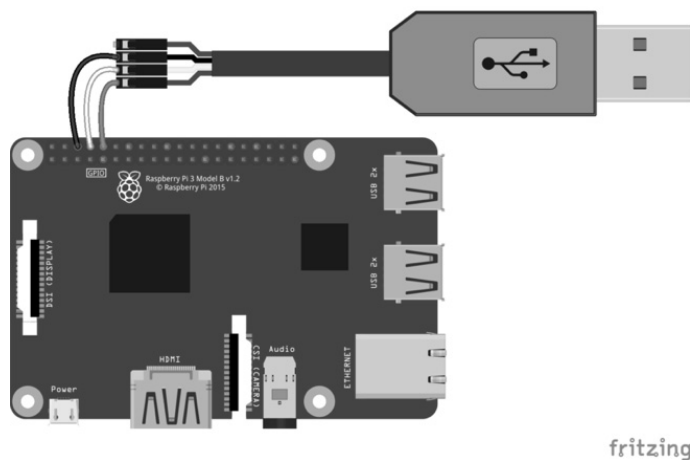


圖 9-35 USB 轉 TTL 板子對接

### Windows PC 電腦上上用 USB 轉 TTL 板子

使用第三方軟體來讀取和送出資料，讀者可以選擇自己熟悉的 UART 軟體，

例如：

- 使用 Hyper Terminal。
- 使用 Putty。

### Windows 上的 Hyper Terminal 安裝

**STEP 1** 確認硬體是否已接好。可以到 Windows\系統\Device Manager\中的 Ports (COM & LPT) 中查看。以筆者的畫面為例，可以看到有個 Usb-to-Serial Comm Port 是使用 com 23，所以請讀者注意自己的 com port 號碼，如果使用的是桌上型電腦，也請確定一下連接的 com port 號碼，應該會是 com 1 和 com 2，請以實際的為主。如果沒有驅動程式的話，常見 PL2303 這個 IC 需要安裝 Windows 驅動程式，網址為 [https://www.prolific.com.tw/US/ShowProduct.aspx?p\\_id=225&pcid=41](https://www.prolific.com.tw/US/ShowProduct.aspx?p_id=225&pcid=41)

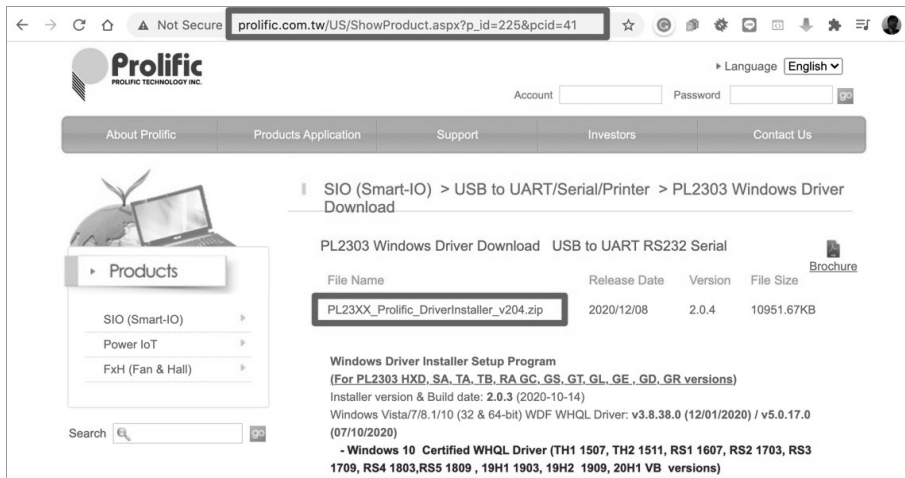


圖 9-36 常見 PL2303 這個 IC 需要安裝 Windows 驅動程式，

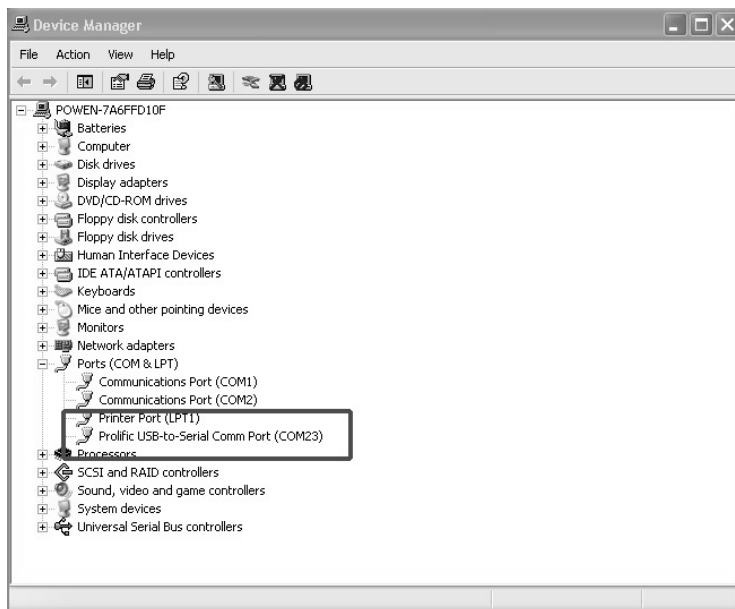


圖 9-37 確定連接的 com port 號碼

**STEP 2** 打開 Windows 中的 Hyper Terminal 軟體。從程式集中的 Accessories\Communications 找到 Hyper Terminal 軟體；在新版的 Windows 已經移除該軟體，可以在網路下載（使用試用版就夠使用了），網址為 <https://www.hilgraeve.com/windows-10-hyperterminal-hyperaccess/>。

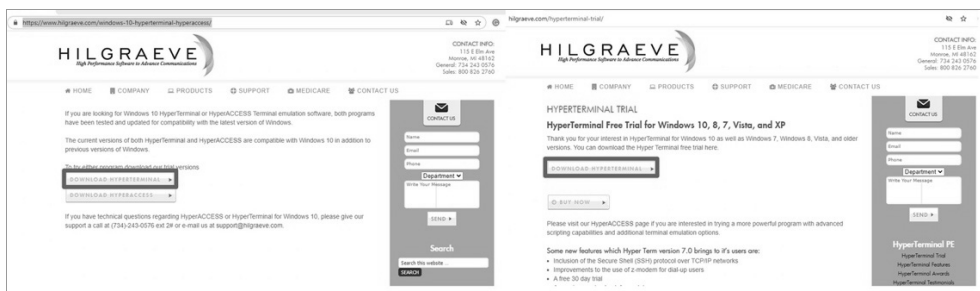


圖 9-38 下載 Hyper Terminal

### 教學影片

安裝 Windows 中的 Hyper Terminal 軟體教學影片請參考  
*9-11-DownloadHyperTerminal.mp4* 影片檔。

## Windows 上的 Hyper Terminal 使用

**STEP 1** 打開 Hyper Terminal 軟體之後，為它設定一個名稱。

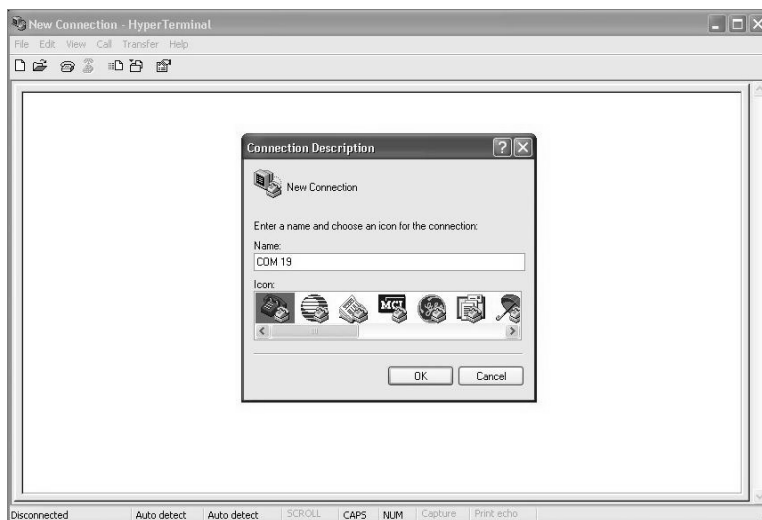


圖 9-39 設定名稱

**STEP 2** Hyper Terminal 軟體會詢問要連結哪一個 com port。在做這個動作之前，請先確定 Arduino 板子已經連到電腦上，並且依照 Step 1 確定連結的 com port 號碼。



圖 9-40 設定 com port

**STEP 3** 設定雙方的傳輸速度，在這裡筆者選用的是：

- Bits per second : 115200
- Data bits : 8
- Parity : None
- Stop bits : 1
- Flow control : None

然後按「OK」鈕確定。

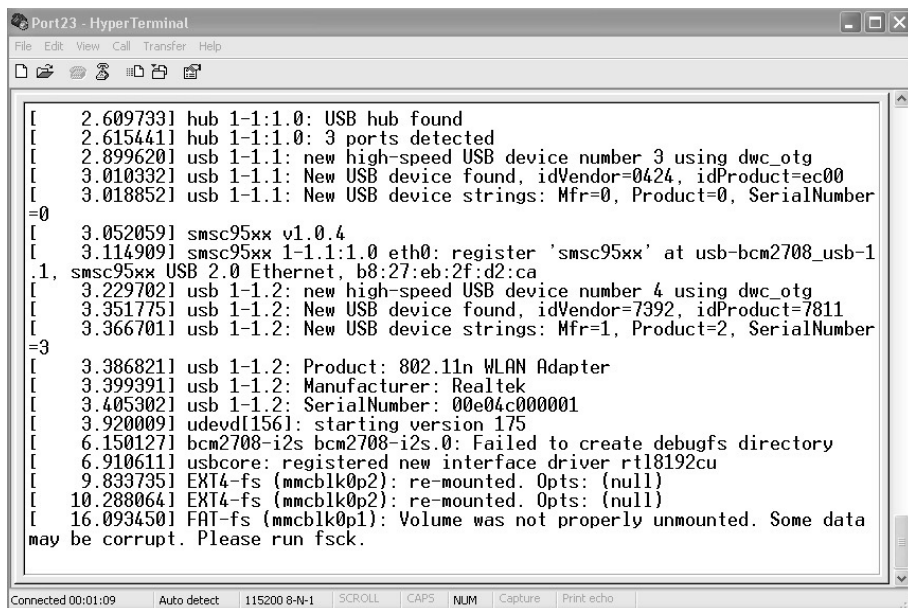


圖 9-41 設定

**STEP 4** 確認 Raspberry Pi 在關機的情況下已經接好硬體接線，並樹莓派重新打開 Raspberry Pi。

## 執行結果

登入帳號：pi，密碼是 raspberry，使用一段時間之後，會發現這就是 Terminal 文字版的樹莓派的作業系統



```
[ 2.609733] hub 1-1:1.0: USB hub found
[ 2.615441] hub 1-1:1.0: 3 ports detected
[ 2.899620] usb 1-1.1: new high-speed USB device number 3 using dwc_otg
[ 3.010332] usb 1-1.1: New USB device found, idVendor=0424, idProduct=ec00
[ 3.018852] usb 1-1.1: New USB device strings: Mfr=0, Product=0, SerialNumber
=0
[ 3.052059] smsc95xx v1.0.4
[ 3.114909] smsc95xx 1-1.1:1.0 eth0: register 'smc95xx' at usb-bcm2708_usb-1
.1, smsc95xx USB 2.0 Ethernet, b8:27:eb:2f:d2:ca
[ 3.229702] usb 1-1.2: new high-speed USB device number 4 using dwc_otg
[ 3.351775] usb 1-1.2: New USB device found, idVendor=7392, idProduct=7811
[ 3.366701] usb 1-1.2: New USB device strings: Mfr=1, Product=2, SerialNumber
=3
[ 3.386821] usb 1-1.2: Product: 802.11n WLAN Adapter
[ 3.399391] usb 1-1.2: Manufacturer: Realtek
[ 3.405302] usb 1-1.2: SerialNumber: 00e04c000001
[ 3.920009] udevd[156]: starting version 175
[ 6.150127] bcm2708-i2s bcm2708-i2s.0: Failed to create debugfs directory
[ 6.910611] usbcore: registered new interface driver rtl8192cu
[ 9.833735] EXT4-fs (mmcblk0p2): re-mounted. Opts: (null)
[ 10.288064] EXT4-fs (mmcblk0p2): re-mounted. Opts: (null)
[ 16.093450] FAT-fs (mmcblk0p1): Volume was not properly unmounted. Some data
may be corrupt. Please run fsck.
```

圖 9-42 開機時，就會看到 Raspberry Pi 傳過來的 Boot 資料

若是沒有修改 Raspberry Pi 的 Raspbian OS 系統，每次開機時，硬體就會透過 GPIO 的 UART 接腳送出資料。如果沒有出來的話，可試著把 RX / TX 的線對調一下，很多時候因硬體廠商的關係，上面印刷的文字可能是相反的，同時確定樹莓派的 Raspberry Pi Configuration/Interface 中的 Serial Port 選項是打開的。

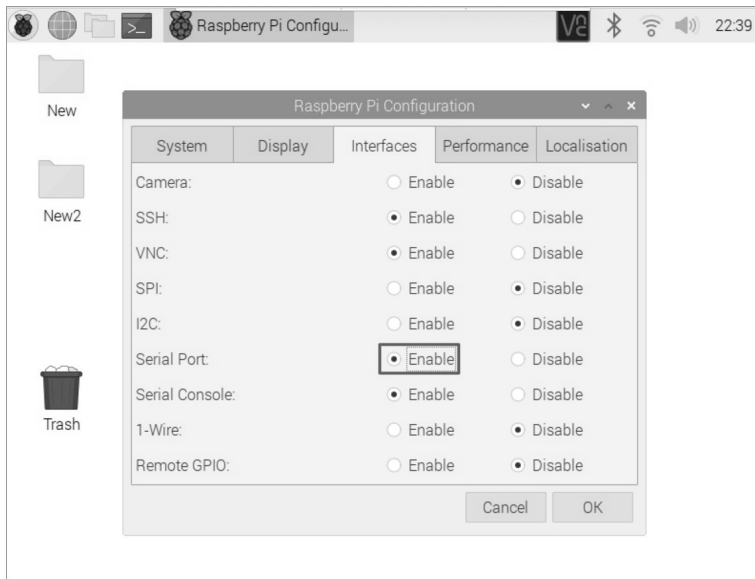


圖 9-43 樹莓派的 Serial Port 的選項是打開 Enable

### 教學影片

請見 9-12\_uart-pc-boot.mp4 影片檔。

## Windows 使用 Putty

**STEP 1** Putty 安裝方法請參考本書〈3.20 Windows 透過 putty 做 SSH 遠端連線〉。

**STEP 2** 打開 Putty 軟體之後。

- 點選 Serial。
- 請依照裝置管理員上顯示的位置，設定連結 COM。
- 速度 Speed 設定為 115200。
- 點選 Open。

就可以順利連接到樹莓派的 TTL。

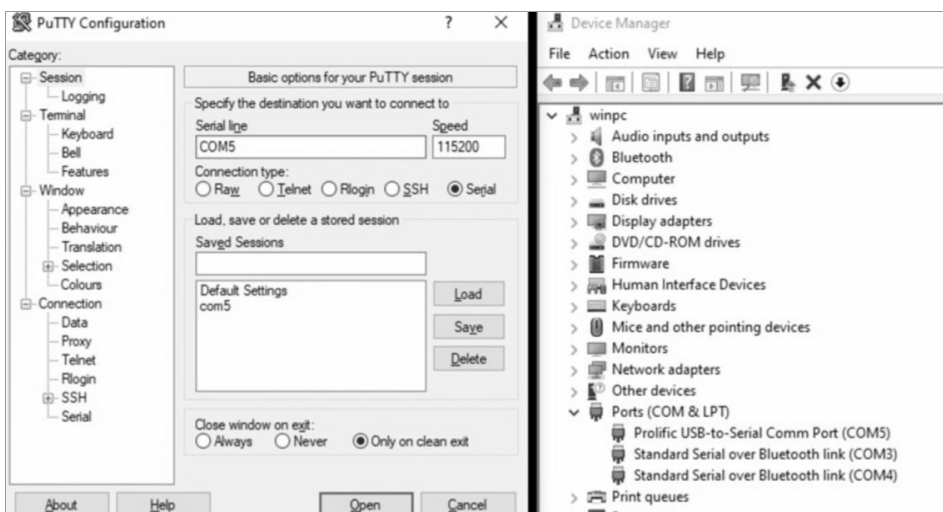


圖 9-44 設定

### 教學影片

請見 *9-13-Putty-USBTTY.mp4* 影片檔。

## 樹莓派 TTL

完成上面的實驗，可以發現作業系統裡面已經安裝好 TTY 連線軟體。

這意味著，上一節的實驗完成後，將 Windows Hyper Terminal 軟體透過上面的方法連接後按下「Enter」鍵，即可連線到 Raspberry Pi。這是因為 Raspberry Pi 的 TTY 軟體是內建的，所以正透過 UART 連線，使用 TTY 在跟 Raspberry 溝通。

## 如何使用 Pin 8/ Pin 10 做一般的資料傳遞？

由於 Raspberry Pi 的系統內定已開啟 TTY，會透過 UART 送出開機訊息，該如何關閉 TTY 軟體（尤其當樹莓派 3b 的 Pin 8 和 Pin 10 都已經藍牙占用時）？

透過以下方法可以將關閉，關閉後如果又需要它，再反做一次回來即可。

**STEP 1** 執行文字編輯器，修改一下設定檔。

```
sudo nano /boot/config.txt
```

在最後添加一段文字

```
dtoverlay=pi3-miniuart-bt
```

請按下「Ctrl+O」鍵儲存和「Ctrl+X」鍵離開 nano 文字編輯系統。

**STEP 2** 關閉 TTY 軟體，以避免 TTY 軟體占用 GPIO 的 UART 接腳。

透過文字編輯軟體

```
$ nano /boot/cmdline.txt
```

在這個檔案中，會看到一段

```
console=serial0,115200
```

修改如下

```
# console=serial0,115200
```

請按下「Ctrl+O」鍵儲存和「Ctrl+X」鍵離開 nano 文字編輯系統。

**STEP 3** 完成設定後，要重新開機設定檔才會正常工作。

```
$ sudo halt
```

不過還是建議使用 USB 轉 TTL 的硬體接在 USB 孔，會比較簡單。

## 9.12

## UART 序列埠資料傳遞——樹莓派和 PC

### ◎ 介紹

本節會介紹 PC 和樹莓派，如何各自使用軟體和鍵盤輸入文字，傳遞到對方和受到對方所傳過來的文字。

這裡 Raspberry Pi 和電腦連接：

- PC 使用 USB 轉 TTLSerial 硬體。
- RPI 使用 USB 轉 TTLSerial 硬體。

### ◎ 硬體準備，PC 使用 RS243 轉 TTL

-  Raspberry Pi 板子。
-  2 個 COM 序列埠 RS232 轉 TTL 的轉換器。
-  麵包板。
-  接線。

### ◎ 硬體接線

在 PC 和樹莓派的 USB 接口，各接一個 USB 轉 TTL 並透過杜邦線，將兩者接在一起。

| Raspberry Pi 端 USB 轉 TTL Serial | PC 端 USB 轉 TTL Serial |
|---------------------------------|-----------------------|
| USB 轉 TTL 板子的 RX                | USB 轉 TTL 板子的 TX      |
| USB 轉 TTL 板子的 TX                | USB 轉 TTL 板子的 RX      |
| USB 轉 TTL 的 GND 接地              | USB 轉 TTL 的 GND 接地    |

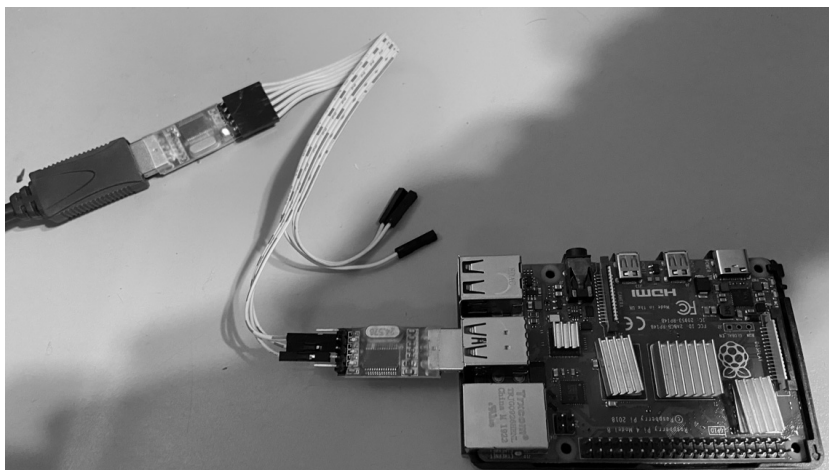


圖 9-45 USB 轉 TTL 板子對接

## 🔗 介紹 minicom

minicom 是一個小型的工具軟體，可以被安裝在 Linux 上的軟體，能透過 serial port 送資料到其他的設備上，其功能如同 Windows 上面 putty 和 Hyper Terminal。

## 🔗 安裝 minicom

### STEP 1 更新安裝程式。

安裝的時候先更新 apt-get。

```
$sudo apt-get update
$sudo apt-get upgrade
```

### STEP 2 安裝 minicom。

安裝方法如下：

```
$ sudo apt-get install minicom
```

```
pi@raspberrypi ~ $ sudo apt-get install minicom
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following extra packages will be installed:
  lrzsz
The following NEW packages will be installed:
  lrzsz minicom
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 420 kB of archives.
After this operation, 1,189 kB of additional disk space will be used.
Do you want to continue [Y/n]? Y
Get:1 http://mirrordirector.raspbian.org/raspbian/ wheezy/main lrzsz armhf 0
.12.21-5 [106 kB]
```

圖 9-46 安裝 minicom

### STEP 3 確認 USB 設備的位置。

請在樹莓派接上 USB 轉 serial 硬體之前和之後各執行一次：

```
$ ls /dev/tty*
```

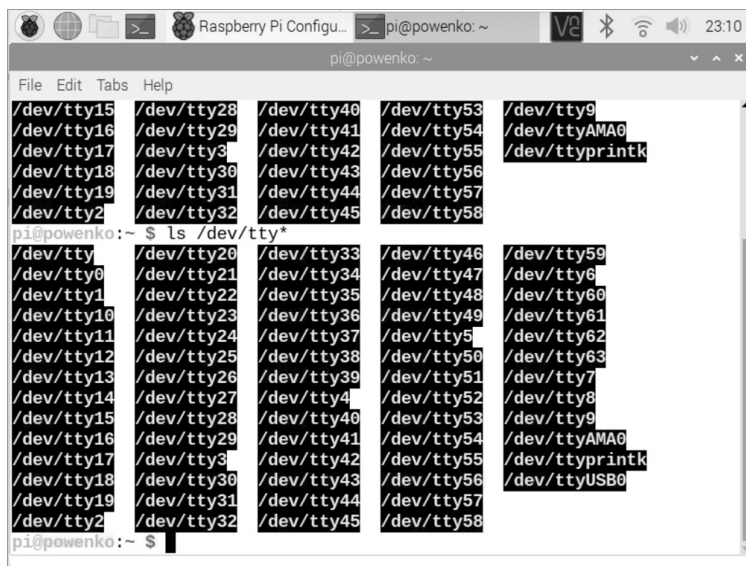


圖 9-47 找設備位置

請注意，當硬體接上 USB 的時候，如果驅動程式認得這個硬體，便會自動產生一個新檔案，有很高的機率會叫做 `/dev/ttyUSB0`，但實際的名字，請讀者自行觀察。

#### STEP 4 連線。

想連線到其他設備時，透過以下的指令即可送出資料。

```
$minicom -b 115200 -o -D /dev/ttyUSB0
```

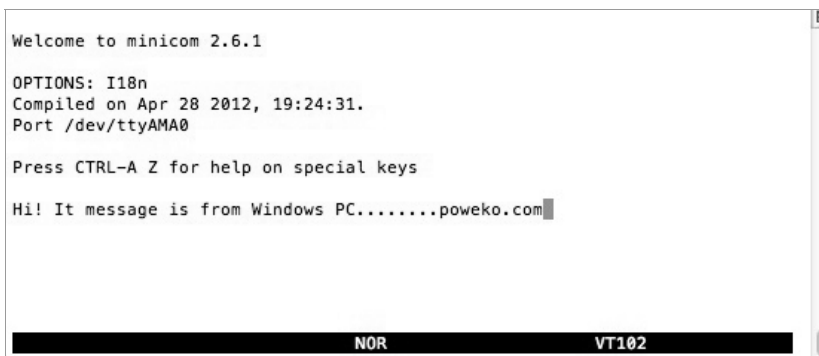


圖 9-48 執行 minicom

## 執行結果

筆者是透過 Windows 的 Hyper Terminal 與 Raspberry Pi 的 minicom 溝通，只要透過鍵盤打字，就會把資料傳遞過去。同樣的，樹莓派和 PC 鍵盤上輸入文字，電腦那邊也會收到所輸入的文字。

樹莓派要離開 minicom，請先按下「Ctrl + A」鍵，再緊接著按下「Z」鍵，會出現 minicom 的選單，這時按下「X」鍵，會詢問是否要離開，選「Yes」鈕就可以離開。

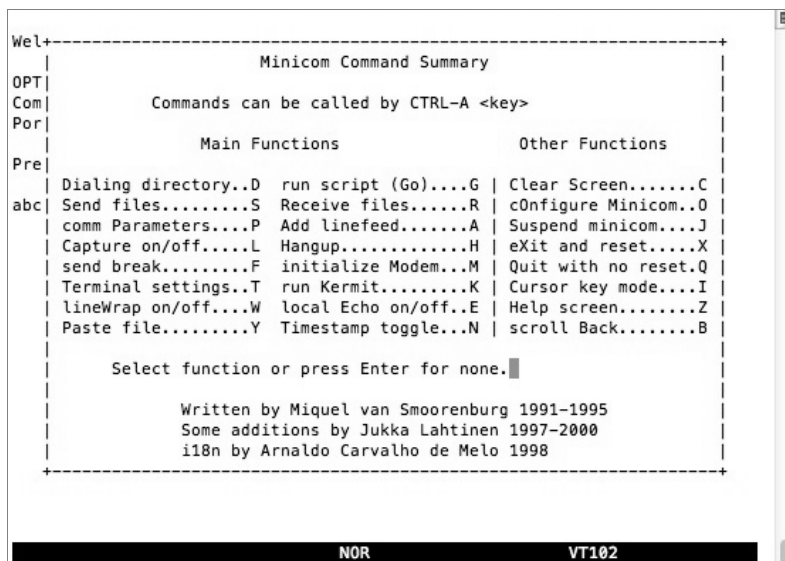


圖 9-49 按下「Ctrl + A」鍵，再按下「Z」鍵就可離開 minicom 視窗

## 教學影片

請見 *9-14-ap-minicom.mp4* 影片檔，該影片介紹如何在 Raspberry Pi 使用 minicom 軟體與 PC 溝通。

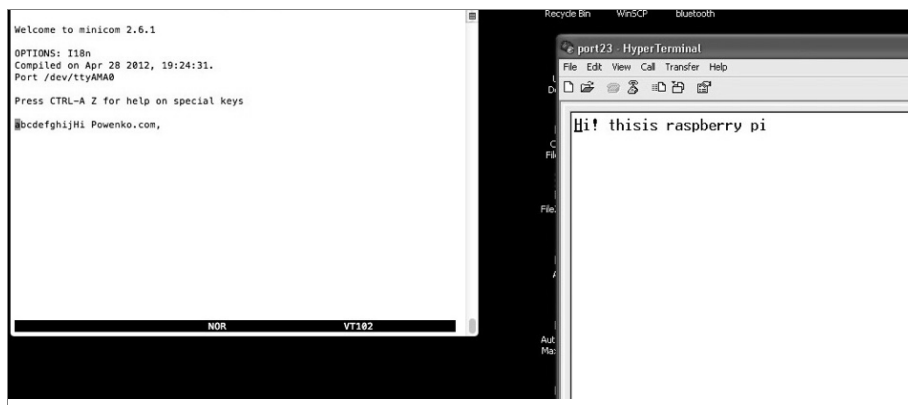


圖 9-50 使用 minicom 軟體與 PC 的 com port 溝通

## 9.13

## UART 序列埠資料傳遞——透過 Python

### ◎ 介紹

讀者想透過 Python 程式來傳遞資料的話，依照下面的步驟完成即可。

### ◎ 安裝模組

在 Raspberry Pi 上要使用 UART 之前，需要先安裝 python-serial 模組。

```
$ sudo apt-get install python-serial
```

```
pi@raspberrypi ~ $ sudo apt-get install python-serial
Reading package lists... Done
Building dependency tree
Reading state information... Done
Suggested packages:
  python-wxgtk2.8 python-wxgtk2.6 python-wxgtk
The following NEW packages will be installed:
  python-serial
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 79.0 kB of archives.
After this operation, 483 kB of additional disk space will be used.
Get:1 http://mirrordirector.raspbian.org/raspbian/ wheezy/main python-serial
  all 2.5-2.1 [79.0 kB]
Fetched 79.0 kB in 1s (71.3 kB/s)
Selecting previously unselected package python-serial.
(Reading database ... 79759 files and directories currently installed.)
Unpacking python-serial (from .../python-serial_2.5-2.1_all.deb) ...
Setting up python-serial (2.5-2.1) ...
```

圖 9-51 安裝 python-serial 模組

透過文字編輯器輸入以下的程式，並存成 `ser.py`（此處筆者是使用 `nano` 文字編輯器）。

```
$ nano 12-ser.py
```

並把程式複製上去。

### 🕒 範例程式：sample\ch09\12-ser.py

```
1. #!/usr/bin/env python
2. # author: Powen Ko
3. import serial
4. import time
5.
6. port = serial.Serial("/dev/ttyUSB0", baudrate=115200, timeout=3.0)
7. port.write("\r\nHi! I am Raspberry")
8. while True:
9.     ch = port.read()
10.    print(ch)
11.    time.sleep(0.1)
```

把程式複製和貼上，透過「Ctrl + O」鍵儲存程式，最後透過「Ctrl + X」鍵離開程式。

### 🕒 程式解說

- 第 6 行：請注意，這裡定義傳輸速度為 115200，並透過 GPIO 上面的 Pin，也就是 `/dev/ttyUSB0`。
- 第 7 行：送出 TX 字串。
- 第 8 行：無盡迴圈。
- 第 9 行：讀取 RX 字元。讀取傳過來的字元。



**注意** 在第 6 行的「`/dev/ttyUSB0`」請依照實際的情況調整設備的位置，如果是要用 Pin8、Pin 10 的話，可以改成「`/dev/ttyAMA0`」，也可以改成「`COM2`」執行在 Windows PC。

## 執行結果

透過以下的指令就可以在樹莓派執行 Python 程式，要離開的時候按下「Ctrl + Z」鍵即可。

```
$ python312-ser.py
```

而 Windows 也可以透過 HyperTerminal 或 putty 執行。

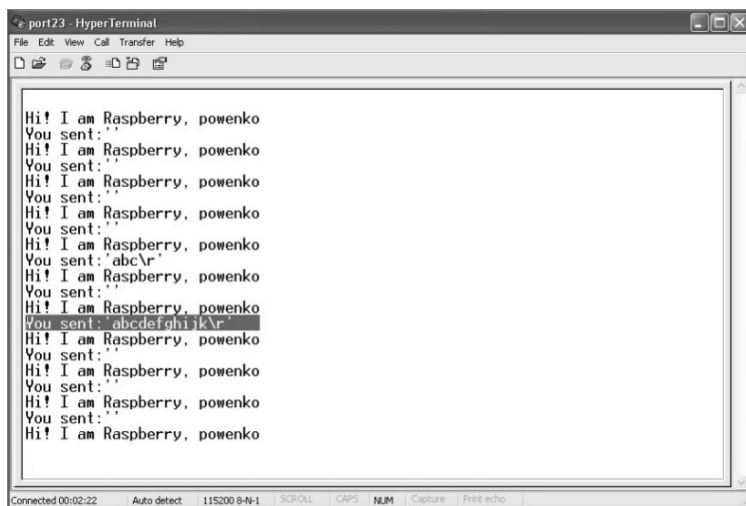


圖 9-52 執行結果

## 教學影片

請見 9-15\_uart-python.mp4 影片檔，影片介紹如何透過 Python 去讀取 UART 序列傳遞資料。

## 補充資料

這一個 Python 的程式可以用在很多地方，可以兩個樹莓派對接，也可以用在 PC 和 PC 之間的連結，只要留意一下 COM 的位置，就可以使用。另外，Arduino 也可以接在樹莓派的 USB 孔上，透過這一個程式就可以讀取和送出的資料。

## 9.14

# Raspberry Pi 的 GPIO SPI

### ◎ 介紹

Raspberry Pi 的 SPI 有系統特定的接腳可以使用，但設定需要特別的模組與動作。而 SPI（Serial Peripheral Interface）是由摩托羅拉公司提出的一種同步串列外部設備，介面匯流排，它可以使 MCU 與各種周邊設備，以串列方式進行通信與交換資訊。匯流排採用 3 條或 4 條資料線進行資料傳輸，常用的是 4 條線，即兩條控制線（選擇目標對象的 CS 和時脈 SCLK），以及兩條資料信號線資料，輸入 SDI 和資料輸出 SDO。

SPI 是一種高速、全雙工、同步的通信匯流排。在摩托羅拉公司的 SPI 技術規範中，資料信號線 SDI 稱為 MISO（Master-In-Slave-Out，主入從出），資料信號線 SDO 稱為 MOSI（Master-Out-Slave-In，主出從入），控制信號線 CS 稱為 SS（Slave-Select，從屬選擇），SCLK 稱為 SCK（Serial-Clock，串列時鐘）。在 SPI 通信中，資料是同步進行發送和接收的。資料傳輸的時鐘，基於來自主處理器產生的時鐘脈衝，摩托羅拉公司沒有定義任何傳輸速度的規定。

### ◎ PI 介面資料傳輸

SPI 是以主從方式工作（Master/Slave），允許一個主設備和多個從設備進行通信，主設備透過不同的 SS/CS 信號線選擇不同的從設備進行通信。當主設備選中某一個從設備後，MISO 和 MOSI 用於串列資料的接收和發送，SCK 提供串列通信時脈，上升就發送，下降就接收。在實際應用中，未選中的從設備的 MOSI 信號線需處於高阻狀態，否則會影響主設備與要傳遞的設備間的正常通信。

序列周邊接口（Serial Peripheral Interface Bus，SPI）又稱串行外設接口，每個設備有 4 條線。

- SS/CS 選取哪個周邊設備。
- MOSI 主機送出訊號。
- MISO 主機接收訊號。
- CLK 時脈。

其中 SS (Slave Select) 是主機 (Master) 選擇向哪個周邊設備 (Slave) 通信，每個不同設備單獨連接一條專用線，信號高表示不啟用，信號低表示啟用。

另外三條線則可以供多個設備共用 (主機應保證通過 SS 專線只選擇一個設備)。

- MOSI (Master out, slave in) 主機發送、周邊設備接收。
- MISO (Master in, slave out) 主機接收、周邊設備發送。
- CLK 時鐘/時脈信號。

## 9.14.1 SPI 設定

**STEP 1** 啟動 GPIO 的 SPI 接腳。

確定樹莓派的 Raspberry Pi Configuration/Interface 中的 SPI 的選項是打開的。

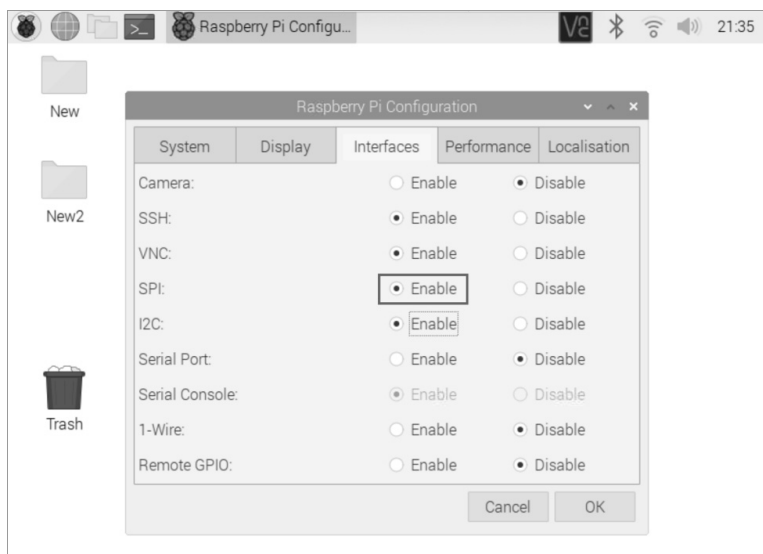


圖 9-53 開啓 SPI 接腳的設定

### SPI 安裝 Python 模組

接下來，需要安裝 Raspberry Pi 的 Python GPIO SPI 的模組，請依照下面的步驟執行。

**STEP 1** 建立下載的文件夾。

因為稍後要由網路上下載檔案，所以先準備一個文件夾以存放資料。

```
$ mkdir py-spidev
$ cd py-spidev
```

**STEP 2** 下載原始程式碼。

請確定網路一切正常，並執行以下的指令。

```
$ wget https://raw.githubusercontent.com/doceme/py-spidev/master/setup.py
$ wget https://raw.githubusercontent.com/doceme/py-spidev/master/spidev_module.c
```

**STEP 3** 安裝。

```
$ sudo python setup.py install
```

## ◎ 實驗介紹

本 SPI 實驗會使用 SPI 的控制方法來控制一顆 ADC IC 來讀取類比資料，並透過 IC 轉換成 4bit 的數位資料。常見的 ADC IC 有：

- ADC0804：有 8 個接腳的數位輸出（9.10 Raspberry Pi Analog 類比輸入介紹）。
- MCP3008：只有 4 個接腳的數位輸出（本章節介紹）。



圖 9-54 MCP3008 的外觀

右圖是 MCP3008 接腳的功能。

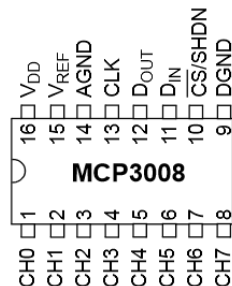


圖 9-55 接腳功能圖

所以 SPI 派上用場了，因為這一顆 IC 就需要使用 SPI 的方法來控制。

本實驗會讀取可變電阻的資料，並透過 MCP3008 的 IC 轉換成數位資料輸入到 Raspberry Pi。

### 硬體準備

-  1 個可變電阻。
-  麵包板。
-  1 個 MCP3008 IC。
-  電線數條。
-  1 個 Raspberry Pi。

### 硬體接線

這個實驗請在沒有接上電源和電池的情況下進行。依照下頁的硬體接線圖，接過一次硬體線路。理論上可以多接幾個可變電阻在 MCP3008 接腳 Pin1～Pin8，因為這個 IC 可以同時讀取 8 個類比 Analog 的輸入。

| ADC0804 接腳 pin  | MCP3008     |
|-----------------|-------------|
| 3.3V            | Pin16/ VDD  |
| 3.3V            | Pin15/ VREF |
| GND             | Pin14/ AGND |
| GPIO11 (pin-23) | Pin13/ CLK  |
| GPIO9 (pin-21)  | Pin12/ DOUT |
| GPIO10 (pin-19) | Pin11/ DIN  |
| GPIO8 (pin-24)  | Pin10/ CS   |
| GND             | Pin9/ DGND  |

🔗 硬體接線圖：sample\ch09\13-spi.fzz

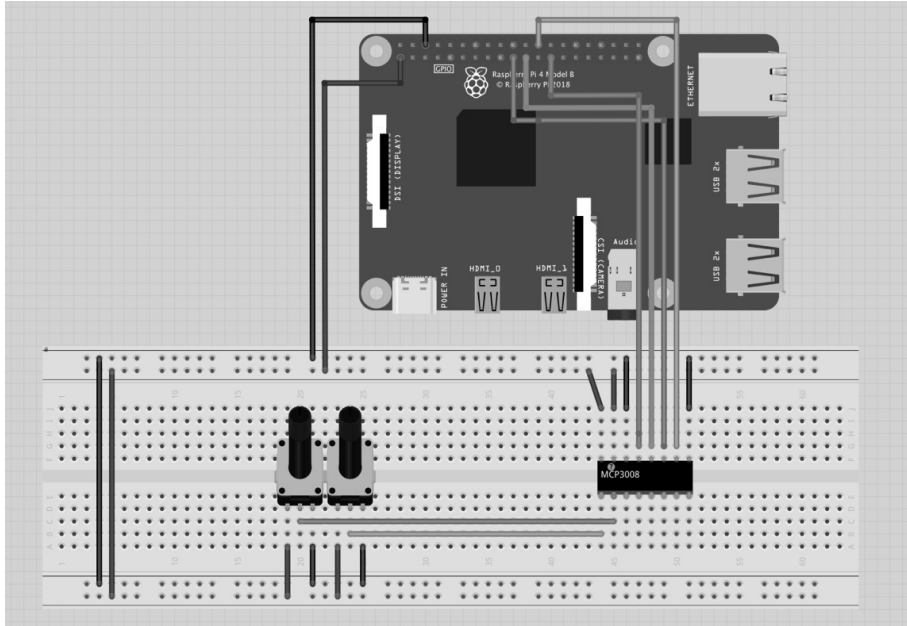


圖 9-56 實際硬體接線

透過文字編輯器輸入以下的程式，並存成 `spicode.py`，此處筆者是用 `nano` 文字編輯器。

```
$nano 13-spicode.py
```

並把程式複製上去。

🔗 範例程式：sample\ch09\13-spicode.py

```
1. #!/usr/bin/python
2. # Author: Powen Ko
3. import spidev
4. import time
5. import os
6. # Open SPI bus
7. spi = spidev.SpiDev()
8. spi.open(0,0)
9. #從 MCP3008 晶片讀取 SPI 資料的函數
10. # 通道必須是整數 0-7
11. def Readchannel(channel):
12. adc = spi.xfer2([1, (8+channel)<<4,0])
```



```
13.     data = ((adc[1]&3) << 8) + adc[2]
14.     return data
15.
16. # 定義傳感器通道
17. while True:
18.     #讀取傳感器資料
19.     data1 = Readchannel(0)
20.     data2 = Readchannel(1)
21.     # 輸出結果
22.     print("value1: {} value2:{}".format(data1,data2))
23.     time.sleep(0.3)
```

把程式複製和貼上，透過「Ctrl + O」鍵儲存程式，最後透過「Ctrl + X」鍵離開程式。

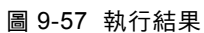
### 🔍 程式解說

- 第 1 行：這是執行環境的定義檔，定義 Python 程式放在哪個路徑。
- 第 2 行：註解的寫法，前面加個# 就可以成為註解。
- 第 5 行：請注意，這裡是透過 spidev 模組。
- 第 17 行：無盡迴圈。
- 第 19 行：讀取第一個 Pin1 的資料。
- 第 20 行：讀取第一個 Pin2 的資料。
- 第 22 行：顯示資料。
- 第 23 行：延遲 0.3 秒。

### 🔍 執行結果

透過以下的指令就可以執行 Python 程式。要離開的時候按下「Ctrl + Z」鍵即可。

```
$ python313-spicode.py
```



請參考 *9-16\_python-spi.mp4* 影片檔，影片介紹如何透過 Python，並經由 SPI 讀取 2 個可變電阻的值。